

中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

题 目： 多核神经网络处理器的计算图切分与调度优化

摘 要：

神经网络处理器需要把一张计算图分配给多个 Core 执行。增加切分数量可以让更多计算同时进行，但也可能增加数据搬运和等待；把相关计算放在同一个 Core，又可能因缓存空间不足而反复搬出、搬入中间结果。因此，本题需要共同决定计算图怎样切分、每部分交给哪个 Core、各部分按什么顺序执行。

本文针对题目给出的三种硬件条件设计调度算法。三种算法都先根据数据依赖生成少量可执行方案，再按完整方案的总体任务执行时间（Makespan）选择输出。

问题一要求每个子图单独执行。若某个子图内有两组运算互不需要对方的结果，而后续运算需要读取这两组结果，算法就将它们划为三个子图，再把前两组中的一组交给另一个 Core，使两组运算具备同时执行的条件。本文给出这种重分配不形成循环等待的条件和证明，并以完整评价判断它是否更快。

问题二允许分配到同一个 Core 的不同子图重复使用保留的数据。算法把生成和读取同一张量的操作放在一起分析，按这些操作分配到的不同 Core 计算边界复制字节数，再结合执行单元的空闲时间调整计算位置和顺序。问题三增加共享只读缓存。对于结果只供唯一后续操作读取的树状计算，先完成的分支需要保留结果，直到后续计算使用。本文分析不同处理次序需要保留的字节数，证明一种在限定模型下使存储峰值最小的排序规则，并用官方评价检验完成时间。

三个固定版本的算法均完成了全部 100 张官方计算图、每图 1~5 个 Core 的评价，每题得到 500 个合法结果。问题一在 2~5 个 Core 上的平均加速比为 1.9516、2.7514、3.4786、4.0553；问题二为 2.3167、3.2356、3.9711、4.5498。问题三在 5 个 Core 上的平均基线加速比为 4.7576；保持调度方案不变，对启用和关闭共享只读缓存的结果逐图比较，平均加速比为 1.0829，按字节统计的逐图命中率均值为 31.36%。问题一 5 个 Core 时的平均求解时间为 3.522 秒，问题二全部测试的平均求解时间为 4.029 秒，均包含方案构造与在线比较。

结果表明，减少搬运字节数或提高缓存命中率，都不足以单独保证完成时间缩短。本文用依赖关系证明方案是否合法，用局部模型指导方案构造，再以官方模拟结果检验性能。全部统计保留同一算法版本的完整结果，并报告搬运增加、候选方案退化及局部证明未覆盖的条件。

关键词： 多核调度；计算图切分；数据搬运；缓存复用；总体任务执行时间

1 问题重述与分析

1.1 本题要解决什么问题

题目要求把一张神经网络计算图交给多个计算核心执行。本文沿用题面的标识，将一个计算核心写作 Core。每个 Core 包含矩阵计算单元 Cube、向量计算单元 Vector、数据搬运单元，以及私有的 L1 和 UB 缓存。所有 Core 共享核外主存。题面把这部分主存简称为 DDR，其名称来自双倍数据率动态随机存取内存（double data rate dynamic random-access memory）。这些硬件对象的容量和带宽由题目固定，算法只能改变计算任务的划分、分配和执行顺序 [1]。

以两次互不依赖的矩阵计算为例，分别交给两个 Core 可以同时执行；但若它们都需要从 DDR 读取同一块数据，就会共同占用 DDR 的总带宽。如果后续计算必须等两次矩阵计算都结束才能开始，还需要计入数据传递和等待。由此可见，均衡计算工作量只是必要的考虑，不能据此直接确定最终完成时间。

题目的主要评价指标是总体任务执行时间（Makespan）：从模拟起点到全部操作完成的时间，即所有操作结束时刻的最大值，单位为时钟周期（cycle）。算法程序在计算机上生成方案所经过的时间以秒计，本文称为求解时间。前者衡量方案，后者衡量程序运行效率。本文的执行时刻均来自官方模拟，不作为真实硬件测量结果。

1.2 怎样读懂输入计算图

题面用张量节点（Tensor）表示一块数据，用操作节点（Op）表示一次计算或搬运。本文采用题面的含义：张量可以是输入、计算的中间结果或最终输出。例如，一个包含四个 16 位数的 2×2 数组占用 $4 \times 2 = 8$ B。这里的 8 B 是数据大小，既不是数组的维数，也不是元素个数。正式计算始终读取输入文件的 size 字段，不从这个示例推测实际用例的数据类型。

有向边（directed edge）用箭头表示数据依赖。 $u \rightarrow t$ 表示操作 u 输出张量 t ； $t \rightarrow v$ 表示操作 v 读取 t 。因此， $u \rightarrow t \rightarrow v$ 要求 u 先生成数据， v 再使用数据。原图只连接操作节点与张量节点，不直接连接两个操作节点。这样的图是二部图（bipartite graph），即节点分成两类，边只连接不同类别。题面还要求它是有向无环图（directed acyclic graph, DAG）：沿箭头前进不能回到起点。

后文为讨论计算之间的先后关系，会另画只包含计算操作的图。例如，把 $u \rightarrow t \rightarrow v$ 表示为 $u \rightarrow v$ 。这张图只用于表达选定的计算依赖，不是原始输入文件，也不能省略数据大小、搬运过程和缓存约束后就声称能够完整预测执行时间。

COPY_IN 和 COPY_OUT 是题面规定的搬运操作。前者把数据从 DDR 搬入 L1/UB，后者把数据从 L1/UB 搬回 DDR。ADD、MUL 等操作则使用指定的计算单元。张量的 id 标识数据节点，操作的 id 标识计算或搬运节点；两者不能混用。

1.3 执行单元怎样同时工作

题面把操作占用的执行管道写作 Pipe，并称其对应资源为硬件流水线（pipeline）。在本文中，硬件流水线（pipeline）特指一个 Core 内处理某一类操作的执行资源及其固定操作顺序：PIPE_M 使用 Cube，PIPE_V 使用 Vector，PIPE_MTE2 和 PIPE_MTE3 处理题面规定方向的数据搬运。官方模拟规定每条 Pipe 在同一时刻至多执行一个操作；不同 Pipe 可以在依赖和资源条件允许时同时执行。本文不描述这些单元内部的取指、译码等电路阶段。

例如，Vector 正在处理已经搬入的数据时，MTE2 可以搬入另一块尚待计算的数据。两项操作若有一段共同的执行时间区间，就发生了计算与数据搬运的时间重叠（overlap of computation and data transfer）。是否能够重叠，取决于输入数据是否可用、执行单元是否空闲、任务是否允许开始，以及 L1/UB 是否有足够空间。官方时间线中的空白只表示没有操作在该资源上执行；没有对应事件证据时，不能进一步断言是哪一项条件造成了等待。

1.4 可以提交什么方案

算法需要决定三件事：每个计算操作属于哪个子图，每个子图分到哪个 Core，以及同一 Core 上的子图执行顺序。提交文件只含 `node_to_subgraph` 和 `core_schedules` 两个字段。前者记录全部非 COPY 操作到子图编号的对应关系；后者为每个 Core 列出子图编号的有序列表。所有待划分操作必须覆盖，每个子图必须恰好安排一次。

执行任务（Task）是主控处理器向一个 Core 一次下发并统一调度的最小执行单元。子图是算法的划分结果，Task 是官方评价程序按场景组织的执行对象。两者的对应关系决定了解题的差别。

题目条件	问题一	问题二	问题三
Task 怎样组成	一个子图对应一个 Task	分配到同一个 Core 的各子图合成一个 Task	与问题二相同
同一个 Core 内的不同子图能否保留数据	必须经过 DDR	可以保留在 L1/UB	与问题二相同
不同 Core 之间如何等待	等待前驱 Task 完成及规定延迟	等待对应 COPY_OUT 完成及规定延迟	与问题二相同，命中缓存也要等待
共享只读缓存	无	无	增加二级缓存（Level 2 Cache, L2）

问题一需要权衡增加并行计算与增加 Task 边界。问题二可以在同一 Core 内重复使用已经保存的数据，但保存时间越长，占用缓存的时间也越长。问题三允许多个 Core 从共享只读 Cache 读取数据；查询时所需数据已在 Cache 中称为命中（cache hit），否则为未命中（cache miss）。因此，请求到达前数据是否已经放入 Cache、是否已经被移除，都会改变读取结果。

图 1.1 用相同的计算片段比较这三种条件，突出数据经过的位置和 Task 组成的变化。

1.5 本文怎样处理这些困难

首先检查依赖是否允许给定的执行顺序。原始图无环并不足够：划分子图、添加同一 Core 的先后顺序，都可能产生新的循环等待。本文对完整方案检查这些关系。

其次，分别估计计算所需的工作量和数据搬运量。同一张量被多个操作读取时，不能简单按操作之间的每条边重复累计搬运。需要查看这些操作被分到哪些 Core，以及是否能够使用已经搬入的数据。

再次，通过完整执行评价比较方案。即使一个局部改动减少了复制字节数，也可能延长等待或改变带宽竞争。本文用简化计算提出可比较的方案，最终仍依据官方模型中的完成时间判断收益。

最后控制求解成本。题面建议避免暴力迭代搜索，以单个用例 5~10 分钟给出结果为佳。本文按输入图的结构构造有限个方案，说明每种改动的条件和停止规则，并把构造、检查、评价和写出方案的时间全部计入求解时间。

2 模型条件与符号说明

2.1 沿用的官方条件

本文不改变原图、官方配置和评价程序。操作的计算结果及原有数据依赖保持不变，算法不能删除计算来缩短执行时间。全部容量、带宽和延迟使用题面固定值 [1]。

每个 Core 的 L1 容量为 524288 B，UB 容量为 131072 B。两者分别管理，不能把未使用的 L1 空间充当 UB 空间。所有访问 DDR 的读写共同分配 60 B/cycle 的带宽。问题三的

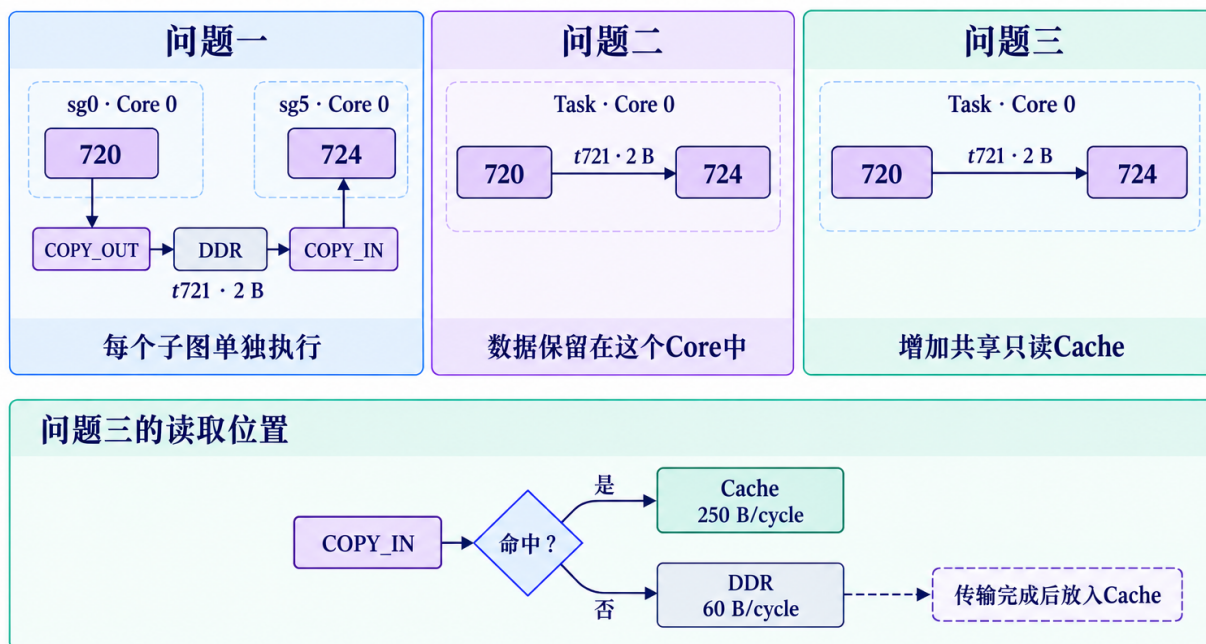


图 1.1 三问条件的差别。相同计算片段在问题一中经过独立 Task 边界，数据经 DDR 中转；问题二允许同一 Core 保留数据；问题三另为符合条件的 COPY_IN 提供共享只读 Cache。下部示意数据能够装入 Cache 时的读取过程；所有请求仍须满足原依赖及规定等待。

共享只读 Cache 容量为 1048576 B，命中读取共享另一组 250 B/cycle 的带宽；这组带宽不占用 DDR 带宽。

问题一在同一 Core 切换 Task 时等待 100 cycles；若 Task 依赖其他 Core 上的 Task，则还须满足前驱完成后 1000 cycles 的条件。问题二、三则检查对应的源 COPY_OUT 完成时刻，目标 COPY_IN 须等待 500 cycles 后才能参与就绪判断。这些常数作用于不同事件，不能在所有跨子图边上重复相加。

执行操作前，官方案程序还检查数据依赖、各 Pipe 的固定顺序、Task 是否允许开始及存储空间。先进先出（first in, first out, FIFO）在本文有两个明确对象：对操作队列，它指按官方预先排定的队列顺序处理；对问题三的 Cache，它指空间不足时先移除最早放入的缓存项，读命中不会改变该项的插入次序。两处顺序不能混为同一个队列。

官方案程序安排一个 Core 中的操作时，如果发现 L1/UB 空间不足，就会插入缓存换出和缓存换入（spill）。需要保留的数据借助 DDR 保存副本，后续计算需要时再重新读入；搬运次数由官方案程序决定。这部分搬运、原始输入输出和不同 Task 之间的数据搬运分别核算，不重复求和。

2.2 原图数据与调度后数据怎样区分

原图的张量 id 对应输入中的一块数据。评价程序插入复制或缓存换入换出后，可能为该数据新建其他张量节点。例如，同一份数值可能同时有 DDR 中的副本和某个 Core 的 L1 副本；内容对应，不代表占用的是同一处存储空间。

官方代码还使用 logical_tid 记录这些副本对应的逻辑数据标识。问题三据此判断请求的数据是否已经存在于 Cache。本文涉及这三种对象时分别写“原图张量节点”“调度

后新建的数据副本”和“Cache 查询标识”，不以一个未限定的名称代替它们。后文的某些数学公式要求原图不同张量标识不合并为同一个逻辑标识；不满足此条件时，不套用相应公式。

本文也区分输入合法、方案顺序无环和官方执行成功。顺序没有循环，并不保证存储容量足够。评价程序拒绝、进程超时和没有运行评价，是不同状态；只有成功执行的结果才有可比较的 Makespan。未取得的结果不填成零参与平均。

2.3 主要符号

符号	本文定义	单位或范围
$G = (O \cup T, E)$	原图； O 为操作集合， T 为张量集合， E 为有向边集合	题面给定
V	原图中全部非 COPY 操作	$V \subset O$
b_t, ℓ_t	张量 t 的字节数及输入规定的存储位置	B；DDR、L1 或 UB
d_v, p_v	操作 v 的计算时长及所用 Pipe	cycle；题面 pipe 字段
K	Core 数量	$1, \dots, 5$
$\pi(v)$	操作 v 所属子图的编号	非负整数
$a(s)$	子图 s 分配到的 Core 编号	$0, \dots, K - 1$
σ_k	Core k 的子图执行列表	有序列表
$P = (\pi, a, \sigma)$	一份完整的切分与调度方案	对应提交文件两字段
q	问题编号	1、2 或 3
$M_q(P)$	方案 P 在问题 q 下的 Makespan	cycle
$D_q(P)$	官方报告的总额外数据搬运量	B
B_i	第 i 图的官方仅使用一个 Core 时的基线 Makespan	cycle
$H_{i,K}$	第 i 图在 K 个 Core 下的 Cache 字节命中率	$[0, 1]$
L, U	最优完成时间的下界和一个可行方案的完成时间	cycle
T_{solve}	从求解程序启动到方案写入文件经过的时间	s

输入的 `cycles` 字段可以为零。推导计算时间下界时，使用与官方规则一致的 $d_v = \max(1, \text{cycles}_v)$ ；不会把输入字段悄悄改写为另一个值。数据单位 B 表示字节，KiB 表示 1024 B。后文公式的新符号在使用处单独定义。

2.4 结果怎样比较

平均加速比（speedup）先对每张图计算基线完成时间与方案完成时间之比，再对全部 100 图取算术平均：

$$\bar{S}_{q,K} = \frac{1}{100} \sum_{i=1}^{100} \frac{B_i}{M_q(P_{i,K})}.$$

其中 $P_{i,K}$ 是算法为图 i 、 K 个 Core 生成的方案。问题一、二要求的主曲线在 $K = 1$ 时按题面设为 1；如果算法还能改善仅使用一个 Core 的方案，其结果另列。基线由官方启发式方法给出，不保证是使用一个 Core 时的最优结果，因此加速比超过 Core 数量并不自动意味着计算错误。

问题三需要保持同一份方案不变，分别启用和关闭共享只读 Cache。记该方案在两种条件下的时间为 $M_3(P)$ 和 $M_2(P)$ ，则 Cache 加速比为 $M_2(P)/M_3(P)$ 。逐图比值的平均，通常不等于两组平均时间之比。

总额外数据搬运量按官方程序计算：调度后的 COPY 字节数减去原图已有的 COPY 字节数。问题三的该字段沿用这一统计方法；Cache 命中后不能自行把相应 COPY 字节改成零，也不能将该字段直接称为命中处理后的 DDR 实测流量。Cache 字节命中率则为命中字节数除以参与查询的总字节数；没有可查询字节时，沿用官方程序取 0。正文报告逐图命中率算术平均，另外说明按访问字节加权的結果。

求解时间 (elapsed time) 采用从程序启动、读取输入到最终方案写入文件的实际经过时间，包含在线产生和比较方案的时间。这里的“在线”指求解当前输入期间发生的工作。独立于求解程序运行的最终官方评价另行计时。

为对应代码，本文用 E0 表示未修改的官方评价程序，用 E1、E2 表示队内用于加速完整方案评分的两个实现标识。这些是代码版本名称，不是性能等级。问题一、二分别在求解过程中使用 E1、E2 比较方案，论文主成绩仍对应 E0 结果；问题三直接使用 E0。它们的调用次数与程序时间分别记录。

3 计算图切分与调度模型

3.1 把三个决策写成一份方案

设 V 为原图中全部需要划分的计算操作。函数 π 为每个操作指定子图编号，所有非空子图组成集合 $\mathcal{S}$ ；函数 a 为每个子图指定 Core；列表 σ_k 给出 Core k 上各子图的先后顺序。三者合在一起构成方案 $P = (\pi, a, \sigma)$ 。仅有节点分组还不是完整方案，因为同一分组采用不同 Core 分配或不同执行顺序，可能得到不同结果。

如果子图 s 中的某个操作需要子图 t 产生的数据，就在子图之间画 $t \rightarrow s$ 。全部这类边组成 E_S 。此外，若同一 Core 的列表中 s 紧接在 t 之后，还需画 $t \rightarrow s$ 表示执行顺序；这些边组成 E_σ 。因此，用于检查先后关系的图为

$$D^+(P) = (\mathcal{S}, E_S \cup E_\sigma).$$

本文称它为“加入同一 Core 顺序后的子图依赖图”，每次使用都指上述集合，不省略其中任一类边。如果图中存在有向环，就有一组 Task 相互等待，方案不能采用。

通过顺序检查后，官方程序按问题编号 q 组织 Task，补充边界搬运，再插入必要的缓存换出、换入和同步关系，最后模拟执行。将这两个过程分别记为 $\mathcal{C}_q$ 和 $\mathcal{R}_q$ ，则

$$P \xrightarrow{\mathcal{C}_q} \mathcal{E}_q \xrightarrow{\mathcal{R}_q} (M_q, D_q, H_q).$$

$\mathcal{E}_q$ 表示组织后的操作、依赖和资源约束， H_q 只在问题三报告。相同大小的两次 DDR 搬运可能因同时存在的其他请求不同而耗时不同，因此不能在这个阶段为每个 Task 预先指定一个与其他 Task 无关的固定时长。

令 $\mathcal{F}_q(G, K)$ 为官方程序能够成功执行的完整方案集合。主要优化问题为

$$\min_{P \in \mathcal{F}_q(G, K)} M_q(P).$$

搬运量 $D_q(P)$ 和问题三的 Cache 命中率另行报告。问题一、二比较方案时优先比较完成时间，时间相同再比较规定的搬运指标；问题三只有在完成时间严格下降时才替换原方案。上述规则是本文的算法选择规则，不是题目给出的加权评分公式。

3.2 三问共用的求解步骤

三问的共同困难在于：便于计算的局部指标不能完整描述机器执行，但对每一种可能划分都进行完整模拟又过于昂贵。本文按以下顺序求解。

第一，从原图识别有明确含义的关系。例如，哪些计算互不依赖，哪些操作读取同一张量，哪些中间结果只供唯一后续操作使用。第二，针对这些关系生成少量完整方案。本文把等待比较的方案称为候选方案（candidate solution）；每份候选都包含切分、Core 分配和顺序。第三，检查覆盖、重复安排和循环依赖。第四，在有证明依据时，用完成时间下界排除无法改善的候选。最后，完整评价剩余方案并选择输出。

图 3.1 给出共同步骤；三条构造分支各自针对不同的输入关系，随后接受同样的完整方案检查。

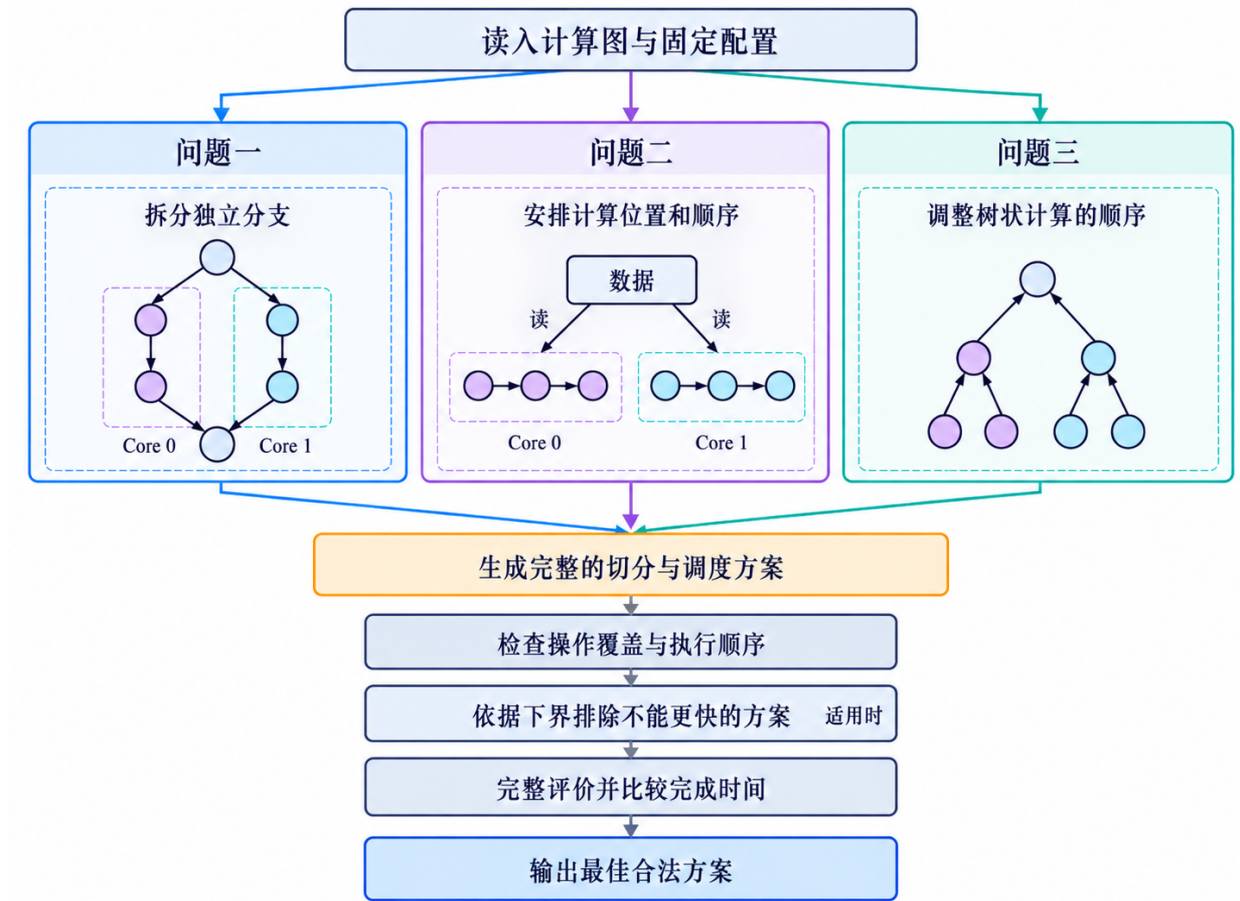


图 3.1 三问共用的方案构造与比较步骤。各分支只在结构条件满足时采用，所形成的完整方案需检查依赖与容量，并通过执行评价比较完成时间。下界只在其证明条件成立时用于提前排除候选。

三问采用的局部计算各不相同。问题一计算重新分配独立分支后各 Core 的工作量；问题二计算生成或读取同一张量的操作分布在多少个 Core；问题三计算先完成的子树留下多少字节。它们分别用于提出方案，不自动等于整个程序的 Makespan。

这一组织方式使数学推导直接对应算法中的决定：无环性证明回答“这个顺序能否采用”，字节公式回答“改动减少了多少边界搬运”，树排序证明回答“在指定模型内应先算哪个分支”。完成时间则由包含全部约束的评价程序确定。不同图不必采用同一个局部改动，但选择规则和评价次数上限在运行前固定。

3.3 为什么局部修改还要检查全图

命题 3.1. 若能给每个子图 s 赋一个整数 $r(s)$ ，并使 $D^+(P)$ 的每条边 $s \rightarrow t$ 都满足 $r(s) < r(t)$ ，则该图不存在有向环。

证明. 假设存在 $s_1 \rightarrow \dots \rightarrow s_m \rightarrow s_1$ ，沿环逐条应用不等式将得到 $r(s_1) < r(s_1)$ ，矛盾。□

这里 r 只是用于证明先后关系的编号函数，不是性能评分。局部重分组时，若能够为新子图安排满足上述条件的编号，就证明了不会引入循环等待；存储容量是否足够、时间是否更短，仍需另外检查。

一个反例可以说明为什么不能只检查每个子图内部。原图有四个操作 a_1, a_2, b_1, b_2 ，只存在依赖 $a_1 \rightarrow b_1$ 和 $b_2 \rightarrow a_2$ ，因此原图无环。若把两个 a 操作合成子图 A、两个 b 操作合成子图 B，则子图关系同时有 $A \rightarrow B$ 和 $B \rightarrow A$ 。两组内部即使没有环，组间也已经形成环。论文中的切分图因而必须保留跨分组箭头。

3.4 完成时间下界能说明什么

下界 (lower bound) 是任何符合所述条件的方案都不能低于的数值。先考虑只统计原始计算的必要条件：每个计算保留原时长，每个 Core 的同一 Pipe 同时至多执行一个操作，并保留能够从官方执行中证明必然成立的计算依赖。

记 W_M 为全部 Cube 操作的时长之和， W_V 为全部 Vector 操作的时长之和，单位均为 cycle。设 L_{cp} 为保留的计算依赖图中路径时长的最大值，即关键路径长度 (critical path length)。因为同一路径上的操作必须依次完成，且每类 Pipe 仅有 K 份同类执行资源，所以

$$L_{\text{basic}} = \max \left\{ L_{cp}, \left\lceil \frac{W_M}{K} \right\rceil, \left\lceil \frac{W_V}{K} \right\rceil, \max_v d_v \right\} \leq M.$$

式中 $\lceil x \rceil$ 表示向上取整。Cube 与 Vector 是不同资源，可以同时工作，因此应分别限制其工作量，不能将 $W_M + W_V$ 除以 K 后作为同一资源的必要时间。依赖图也只能保留有依据的计算关系；经过原始 COPY 的路径不能任意删除中间节点后就当成直接计算关系。

另一类下界针对已经固定的候选方案 P 。此时 Core 分配、各 Pipe 顺序及某些不同 Core 之间传递数据所需的等待已经确定，可以得到更强的 $L(P)$ 。若已有成功方案的时间为 U ，且 $L(P) \geq U$ ，那么 P 不可能严格更快，可以不再对它作完整评价。它只排除该候选，不代表所有可能方案的最优时间都大于 $L(P)$ 。

只有当 L 限制的是同一完整优化问题、 U 来自该问题的一个可行方案时，才有 $L \leq \text{OPT} \leq U$ ，其中 OPT 表示最小完成时间。于是

$$\frac{U}{\text{OPT}} - 1 \leq \frac{U}{L} - 1.$$

例如， $U/L \leq 1.05$ 能够证明该方案距最优不超过 5%。反过来，比值较大可能是算法还有改进空间，也可能只是下界忽略了较多约束，不能把这部分差值都当成可以实现的收益。

3.5 怎样控制程序运行时间

设解析和构造方案的总计算量为 $C(G, K)$ ，候选数量为 b ，第 j 次完整评价耗时为 T_j 。操作之间保留的依赖集合记为 E_V 。覆盖检查和拓扑检查可用与节点、边数量成正比的时间完成，因此总求解时间可写为

$$T_{\text{solve}} = O(C(G, K) + b(|V| + |E_V| + |\mathcal{S}|)) + \sum_{j=1}^b T_j + T_{\text{IO}},$$

其中 T_{IO} 为读入和写出所需时间。这个式子把完整评价的成本单独保留，因为一次评价的耗时会随图规模和存储事件数变化。后文分别给出三种算法的构造条件、评价次数上限和实际求解时间。

进一步减少中间表示的信息量，需要证明压缩前后能够判断相同的操作、得到相同的后续结果。仅有当前完成时间相等还不够。附录给出这一要求的形式化说明；本文没有把这种一般压缩方法列为已经实现的算法。

4 问题一：子图分别执行时的切分与调度

4.1 拆分一个 Task 有哪些收益和损失

问题一中，一个子图构成一个 Task。拆分能够把其中一部分计算交给另一个 Core，但新 Task 之间的数据必须经过 DDR；即使它们最终仍分到同一个 Core，也不能保留跨 Task 数据。因此，算法需要判断新获得的并行计算是否值得新增的搬运和等待。

设 Task s 在 Core k 的开始、完成时刻分别为 A_s, F_s 。记 $\text{prev}(s)$ 为同一 Core 上紧邻它的前一个 Task， $\text{pred}(s)$ 为它所依赖的 Task 集合。题面至少要求

$$A_s \geq F_{\text{prev}(s)} + 100,$$

$$A_s \geq F_t + 1000, \quad t \in \text{pred}(s), \quad a(t) \neq a(s).$$

不存在的前驱不参与约束，时刻单位为 **cycle**。这些不等式说明拆分会增加哪些开始条件，却不能单独算出 $F_s - A_s$ ：Task 执行期间还可能与其他 Core 争用 DDR 带宽。仅把操作周期相加，也不足以得到其最终持续时间。

4.2 先按依赖结构生成方案

算法首先识别计算图中彼此没有计算依赖的部分。把有依赖的操作间连线暂时看作无向边，能够沿连线互相到达的操作构成一个连通分量（connected component）。不同连通分量可以分别安排，但若它们读取同一外部输入，搬运仍可能相互影响。

固定实现最多产生六种基础方案，分别考虑：整体分配这些独立部分；处理工作量集中的较大部分及末端汇合计算；拆分工作量过多的部分；安排读取共享输入的计算；分开能够独立计算的分叉支路；处理矩阵计算之后经过一段向量计算、再进入矩阵计算的受限链状结构。最后一类只在中间结果不被其他分支读取、并满足容量检查时采用。这里逐项说明构造依据，不把六种情形写成六个无需解释的机制名称。

在基础方案上，还可按固定条件增加三类比较：调整上述受限链的分组长度；尝试两种保持局部依赖结构的计算顺序；把符合条件的一个独立分支改分到另一 Core。六份基础方案、至多一份分组长度调整方案、至多两份顺序调整方案和至多一份分支重分配方案，使在线 E1 评价总数不超过十次。结构不满足条件或完整方案重复时跳过，不为凑足十次而继续计算。

E1 根据完整方案评分，用于当前输入的方案选择；主结果由对应的 E0 原件确认。实现还会在编译输入相同、相关初始状态满足条件时重复使用已算出的中间结果。这只能减少重复工作，不能据此宣称任意不同方案在所有后续修改下等价。

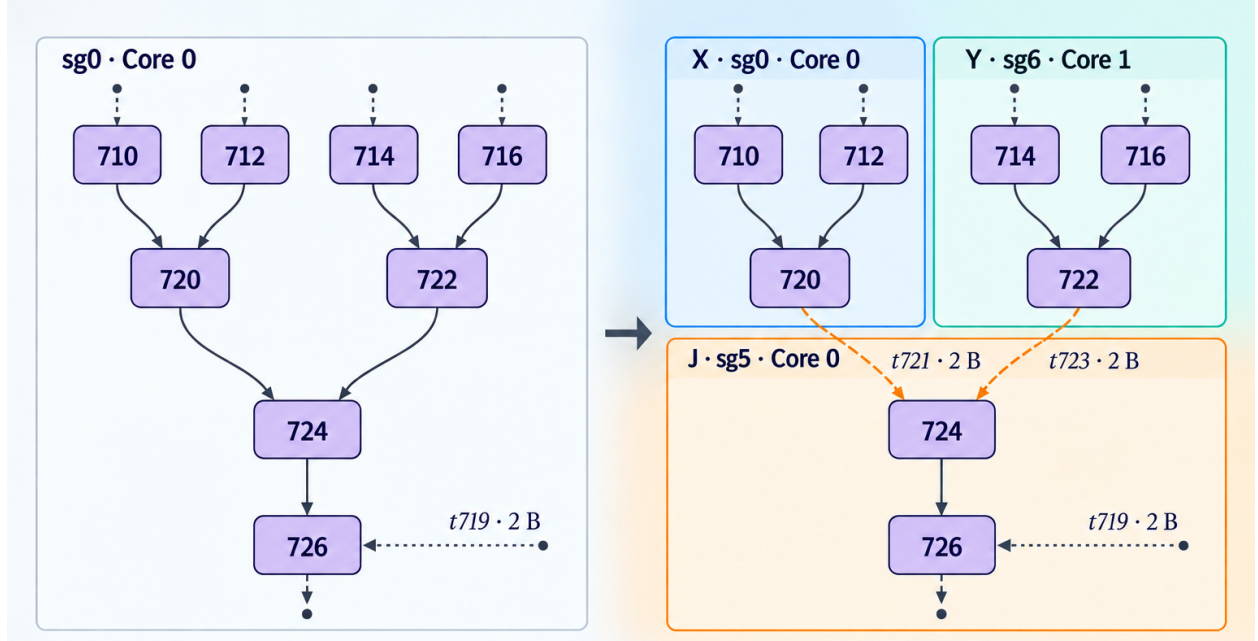


图 4.1 case_026、5 Core 的真实归约片段。左为原始划分，右为将独立支路改分到另一 Core；八个 ADD 节点的编号和依赖不变。点线端口连接裁剪范围外的输入或后继，橙色虚线为位于不同子图中的操作之间的依赖。

4.3 把独立分支交给另一个 Core

以下说明上一节最后一类改动。对原方案的子图依赖图，定义 $h(s)$ 为从起点到 Task s 的最长依赖路径边数；没有前驱时 $h(s) = 0$ 。算法只处理同一 Core 上的 Task 按 h 严格递增排列的方案，并要求同时参与改动、具有相同 h 的不同 Task 之间没有计算依赖。

在工作量较大的一个 Task 内，找到两条互不依赖、随后汇合的计算支路。把相关操作划为三个互不重叠的集合： Y 为准备交给另一 Core 的独立支路， X 为留在原 Core 的其余前部计算， J 为等待两侧结果的汇合计算及其后续。要求 X 与 Y 之间没有依赖， J 不能返回依赖到 X 或 Y ，与外部 Task 相连的边仍遵守原来的 h 先后关系。

原来承担该 Task 的 Core 依次执行 X, J ；接收 Y 的 Core 先执行 Y ，再执行其原有的、具有相同 h 值的 Task。因此，程序必须确认接收 Core 原先存在这个 Task，而且在同一次修改中不会再从接收 Core 向其他 Core 转出支路。算法根据各 Core 的计算工作量，估计重新分配后最大的工作量是否严格减少。只有满足此条件，才继续生成需要完整评价的方案。

命题 4.1。在上述条件下，保留不同 h 之间的原顺序后，这种分支重新分配不会在加入同一 Core 顺序后的子图依赖图中引入有向环。

证明. 先按 h 的大小排列 Task。对 h 相同的 Task，将独立的 X, Y 放在前面，将接收 Core 原有的 Task 放在收到的 Y 之后，将 J 放在它所需的 X, Y 之后；若接收多个 Y ，再依实际次序编号。原方案中这些 h 相同的 Task 互无计算依赖， X, Y 也互不依赖， J 没有返回边，且接收 Core 不同时向外转出支路。因此，在每个固定 h 内，数据依赖及同一个 Core 中的执行顺序都与这个编号一致。连接不同 h 的边则严格增加 h 。由命题 3.1，所得图无环。□

这个证明只说明执行顺序不会形成循环。新增子图仍可能增加复制，改变存储申请和操作等待。因此，形成完整方案后仍需检查全部操作覆盖与顺序，再用 E1 比较完成时间及代码规定的 COPY 统计量。

4.4 算法步骤与停止规则

算法 1：问题一的切分与调度。

输入：原图 G、Core 数量 K、官方固定配置。

1. 按已固定的六类结构条件产生基础方案，删除内容完全相同的重复方案。
2. 比较需要评价的基础方案，保留当前最优的完整方案。
3. 若受限链状结构满足要求，增加一次分组长度调整并比较。
4. 按固定的两种规则分别调整局部计算顺序，检查并比较所得方案。
5. 若满足本章定义的高度与依赖条件，构造一次 X、Y、J 分组及分支重新分配。
6. 检查全部操作覆盖、每个子图只安排一次，以及完整顺序无环。
7. 仅按规定的完整评分接受改动，输出最佳合法方案；总 E1 调用不超过十次。

读取、覆盖检查和拓扑检查的时间与图中节点、边数量成正比；按工作量排序和查找可重分配支路还会增加额外计算，不能把整个算法称为一次线性遍历。分支重新分配只对 Task 数不超过 320 的方案尝试，结构检查设置 2 秒软预算：程序在规定检查位置发现超时后退出，并不保证在任意指令处都能立即中止。该分支至多增加一次 E1 评价。调整分组长度的过程另有编译和子进程时间限制，均计入总求解时间。

4.5 全部用例的结果

固定版本在 100 张图、每图 1~5 个 Core 的 500 种组合上均得到合法结果。下表同时列出题目要求的主曲线与算法改善仅使用一个 Core 的方案后的比值，避免混用上述两个在一个 Core 条件下的定义。

Core 数量	1	2	3	4	5
官方口径平均加速比	1.0000	1.9516	2.7514	3.4786	4.0553
优化方案 $\text{mean}(B/M)$	1.0021	1.9516	2.7514	3.4786	4.0553
改善/持平/退化	0/100/0	8/92/0	15/85/0	12/88/0	13/87/0
求解时间均值 / s	0.436	3.316	3.426	3.427	3.522

5 个 Core 时，平均加速比由前版 4.0259 提高到 4.0553，相对增幅为 0.729%。分支重新分配在全部 500 种组合中被选用 36 次，其余保留此前方案。由于完整版本同时包含其他固定规则，不能把表中全部改善都归因于这一项改动。

5 个 Core 时的额外 DDR 搬运总量为 1053684536 B，比前版增加约 0.389%。因此这次版本改进缩短了完成时间，但没有同时减少总搬运。求解时间的 95% 分位数为 16.623 秒，最大为 25.552 秒。该批次在共享主机上用四个工作进程执行，没有清空操作系统缓存；这些时间不代表独占机器上的确定延迟。

全部测试包括 500 次新求解进程和 1130 次在线 E1 评价。48 种组合进行了新的 E0 评价；另外 452 种组合在方案文件、原图、Core 数量、配置和评价规则逐项一致后，引用已有的 E0 结果。旧 E0 的运行时间没有记为新测值。

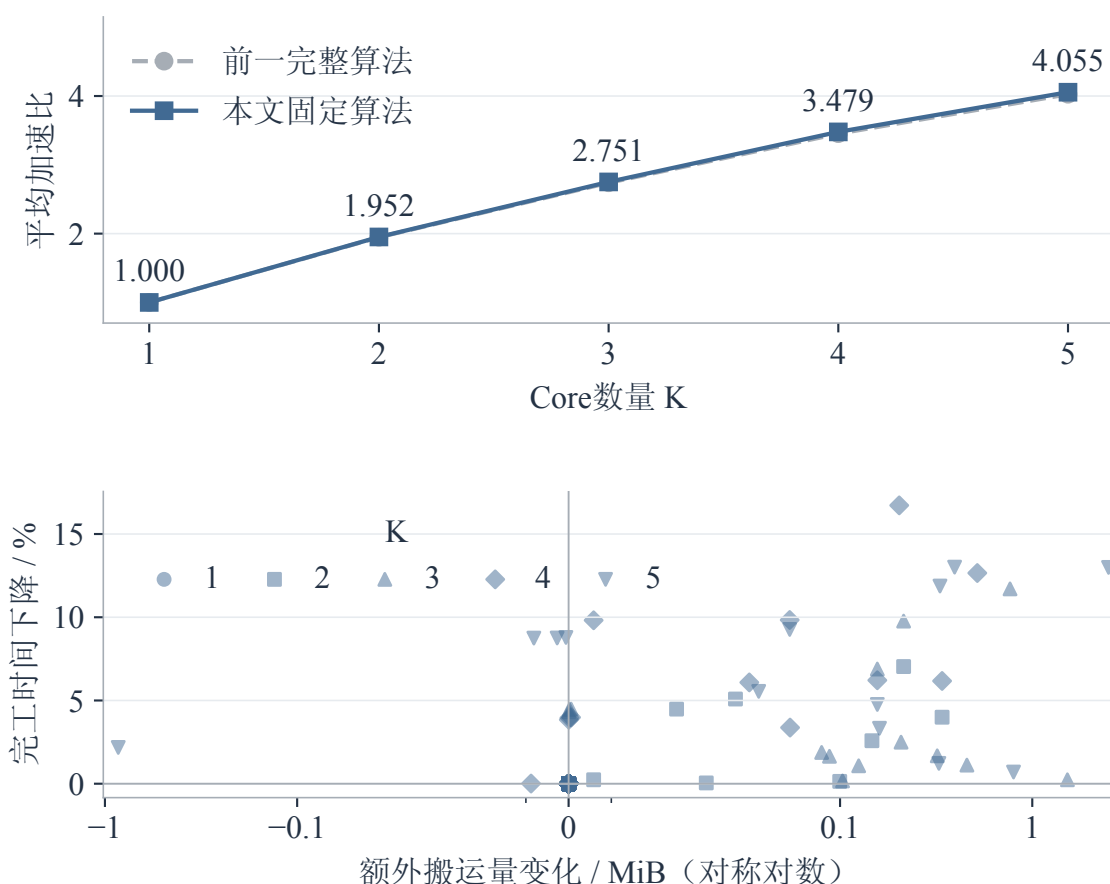


图 4.2 问题一的完整版本比较。上：每个 Core 数量的 100 图平均加速比；下：全部 500 种图与 Core 数量组合的时间与搬运量配对变化。Core 数量用点形区分，横轴采用对称对数。

图 4.3 将修改前后的完整方案放在相同时间尺度上比较，可直接观察各 Core 的结束时刻和不同 Pipe 的执行区间。

这个例子展示了问题一的取舍：把独立计算交给另一个 Core 能够减少串行工作，但也要付出新的搬运和等待。下一章允许同一个 Core 保留在不同子图之间传递的数据后，需要重新计算这种取舍，不能直接沿用问题一对每个 Task 边界的处理。

5 问题二：联合安排计算位置与数据搬运

5.1 同一 Core 可以保留在不同子图之间传递的数据

问题二把被分配到同一个 Core 的各个子图合并为一个执行任务 (Task)。前面子图生成的数据，可以留在这个 Core 的 L1/UB 中，供后面子图中的操作使用。如果生成和读取数据的操作分配到了不同 Core，数据仍须通过 COPY_OUT 写入 DDR，再由目标 Core 的 COPY_IN 读入；目标读取须等待源 COPY_OUT 完成后 500 cycles。此时，把每个计算操作单独划成子图，也不意味着为每个操作建立独立 Task。提交文件中的子图顺序会影响官方程序在这个 Core 中安排操作的先后次序。

这种变化使分配与排序更加相关。把读取同一张量的操作安排到同一个 Core，可能只需搬入一次；但如果两次使用相距较远，这块数据就要保存较长时间，期间可能妨碍其他数据分配空间。把这些操作分到不同 Core 虽然增加搬运，却可能让计算更早完成。本文分别估计可安排计算的时间和边界复制字节数，再比较完整方案。

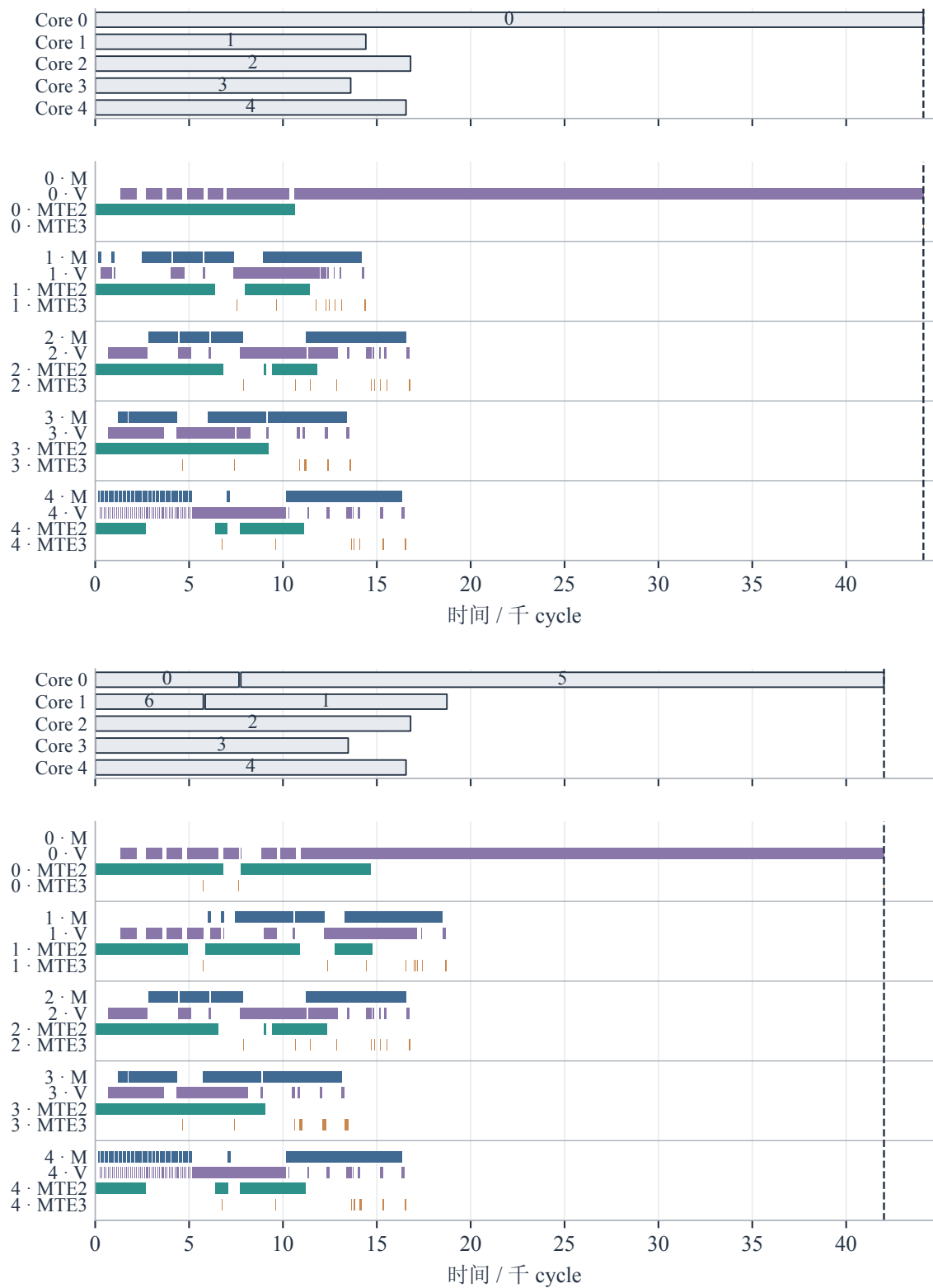


图 4.3 case_026、5 Core 两种切分的执行时间线。上：修改前方案， $M = 44,114$ cycles，额外搬运 211,680 B；下：分支重新分配后， $M = 42,014$ cycles，额外搬运 375,532 B。每组分别显示 Task 持续区间及四条 Pipe 的操作区间，横轴范围相同，数据来自官方模拟事件。

5.2 把计算放进足够长的空闲区间

算法记录每个 Core 上 Cube 和 Vector 已经安排的计算时间区间。安排下一个操作时，先根据保留的计算依赖估计它最早能够开始的时刻，再寻找同类执行单元上足够长的空闲区间。这样，操作可以插到两次已安排计算之间，不必总是排到最后。

设 v 的估计最早开始时刻为 r_v ，计算时长为 d_v ； $\mathcal{I}_{k,p_v}$ 是 Core k 的 Pipe p_v 已经占用的时间区间的并集。选取最早的 s_v 使

$$s_v \geq r_v, \quad [s_v, s_v + d_v) \cap \mathcal{I}_{k,p_v} = \emptyset.$$

区间采用左闭右开表示，因此一次操作恰好在另一次完成时开始，不算同时占用资源。两条计算支路随后汇合时，算法联合比较它们的 Core 分配，至多检查 K^2 个 Core 组合，避免只看一侧较早完成而忽略另一侧。

这个估计没有完整模拟 DDR 带宽分配、缓存换入换出和存储空间分配次序。因此 s_v 只是生成方案时使用的估计，不能画成官方模拟时间，也不是完成时间的必然上界或下界。算法最终用单操作子图及其列表顺序提交结果，由官方程序决定实际开始时刻。

同一个 Core 保留数据与在不同 Core 之间复制数据的区别见图 1.1。后续按这个区别计算边界复制量。

5.3 按同一张量涉及的 Core 集合计算复制量

先看一个简单例子。计算操作 u 生成 8 B 的张量 t ，操作 v_1, v_2 都读取 t 。如果两个读取操作分到同一个远端 Core，就应按该 Core 需要的数据副本统计，而不能把两条读取关系视为互不相关的两次向另一个 Core 传递数据的需求。若只把 v_1 移回 u 所在的 Core， v_2 仍需远端副本，相关边界复制量未必下降；将两个读取操作一起移回后，才消除这份远端需求。

为表示这种多操作共同使用一块数据的关系，引入超边 (hyperedge)：它是一个顶点集合，可以同时包含两个以上顶点；由顶点与这些集合组成的结构称为超图 (hypergraph) [3][5]。本文以计算操作为顶点，对中间张量 t 建立集合 $e_t = \{u, v_1, v_2, \dots\}$ ，其中包含生成 t 的计算操作和所有读取 t 的计算操作。超边只表示共同关联，不表示执行方向；原来的有向依赖仍单独保留。

应用以下公式前要求：全部非 COPY 计算操作已完整纳入模型，原图不同张量标识不通过逻辑标识合并，每个中间张量至多由一个原始计算操作生成，张量字节数为非负整数。这里的“生成”指计算得到数据，不把后来 COPY 得到的副本当成另一次原始计算生成。

给定 Core 分配 a ，定义 $\lambda_e(a)$ 为超边 e 中各操作所在的不同 Core 数量。令 w_e 表示新增一个相关 Core 所增加的边界复制字节数， C 表示与分配无关的固定复制量， $\mathcal{H}$ 为全部需计入的超边集合。在官方 Step2 插入容量不足所需的缓存换出、换入之前，复制量为

$$D_{\text{pre2}}(a) = C + \sum_{e \in \mathcal{H}} w_e (\lambda_e(a) - 1).$$

式中 D_{pre2}, C, w_e 的单位都是 B， λ_e 是无量纲整数。下标 pre2 只表示官方 Step2 之前的计算位置，不是另一种硬件性能指标。

对 DDR 中的外部输入，只把读取该数据的计算操作放入相应集合。每增加一个需要该输入的 Core，就增加 b_t B 的搬入量，所以 $w_e = b_t$ ，第一次输入复制计入 C 。对由计算产生的中间张量，集合还包含生成它的操作；官方边界构造对每个远端 Core 计入输出、输入一对复制，所以 $w_e = 2b_t$ 。最终输出等与分配无关的部分也计入 C 。权重必须按这两类来源分别计算，不能给所有依赖统一赋值。

这说明前述 8 B 示例中，当 u 在 Core 0、两个读取操作在 Core 1 时， $\lambda_e = 2$ ，该中间张量贡献 16 B 的边界复制。只移回一个读取操作，仍贡献 16 B；两个都移回后， $\lambda_e = 1$ ，该项变为 0。这里没有说整个方案的搬运量为 0，也没有把 16 B 换算成未经评价的时间收益。

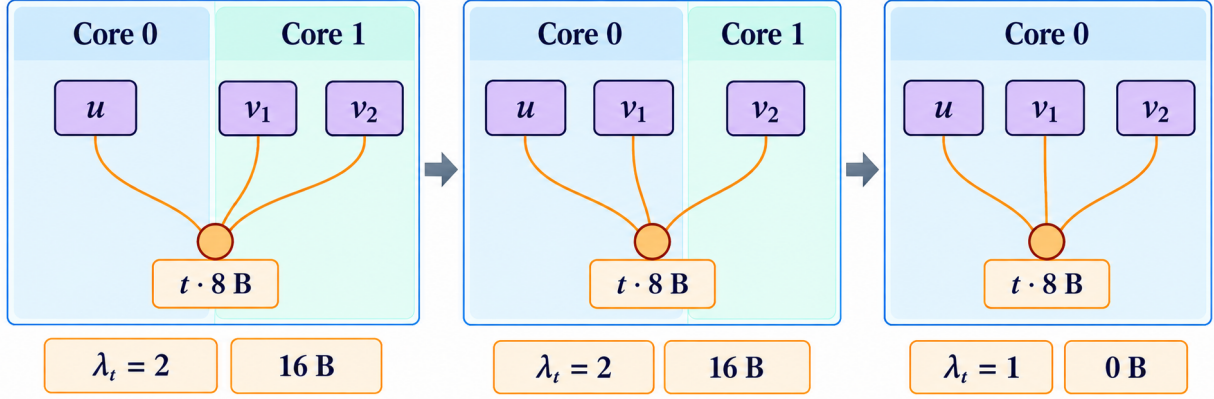


图 5.1 同一张量的计算操作分配。图中 u 生成 t ， v_1, v_2 读取 t ； λ_t 表示这三个操作所在的不同 Core 数量。16 B、16 B、0 B 仅统计该张量的边界复制，线条只表示集合关联。

还可以只更新受移动影响的超边。设一组操作 A 全部从 Core k 移至不同的 Core l 。对与 A 相交的超边 e ，令 $n_{e,k}, n_{e,l}$ 为移动前两 Core 中属于 e 的操作数量。若目标 Core 原先没有相关操作，则增加一次权重；若源 Core 的相关操作全部被移走，则减少一次权重。因此

$$\Delta_e = w_e (\mathbf{1}\{n_{e,l} = 0\} - \mathbf{1}\{n_{e,k} = |e \cap A|\}).$$

$\mathbf{1}\{\cdot\}$ 在括号条件成立时为 1，否则为 0；没有与 A 相交的超边变化为 0。这个式子能够计算字节数变化，却仍未包括缓存容量不足引发的额外搬运，也没有描述计算与搬运能否同时进行。

5.4 限制计算工作量后重新分配局部操作

算法在上一节的时间区间构造结果上，选择两个 Core 及最多 16 条计算链组成局部调整范围。计算链指按依赖前后相接的一串计算操作；范围之外的操作以及必须保持位置的链不允许更换 Core。其余操作在所选两个 Core 之间选择位置。

对于固定范围与固定操作，目标是减少这一范围影响的边界复制量。借助辅助节点，可以把两种 Core 选择表示为源点与汇点之间的最小割问题（minimum source-target cut）：在一个带非负边权的有向网络中，将指定的源点和汇点分到两侧，并使从源点一侧指向汇点一侧的边的权重和最小。辅助节点把“某条超边是否同时涉及两侧”转换为相应边权，因此求得的是当前局部复制字节目标的精确最小值。

只限制字节数可能把过多计算集中到同一 Core。实现因而还规定每个 Core、每类计算 Pipe 的累计计算时长上限。若新的分配超过上限，就固定部分移动链回到原 Core，再求解缩小后的局部问题。原分配始终保留为可行选择；若局部有 r 条可调整链， $r \leq 16$ ，至多求解 $r + 1$ 次。接受前重新计算工作量与复制量，两项检查都通过才保留改动。

命题 5.1. 若每次固定操作都与原分配一致，每次最小割得到当前局部问题的精确最优值，且接受前核对计算时长上限，则被接受方案的局部 D_{pre2} 不大于原分配。

证明. 原分配在每次增加固定操作后仍可行，因此每次局部最优值都不大于原分配值。最终只接受满足计算时长上限且重新核算复制量不增加的方案，结论成立。□

命题不保证同时带全部工作量约束的全局最优，因为把更多操作固定在原 Core 会缩小选择范围；它也不保证 Makespan 下降。算法保留调整前的完整方案，让最终评价决定是否采用新的分配。

5.5 完整算法及计算量

算法 2：问题二的切分与调度。

输入：原图 G 、Core 数量 K 、官方固定配置。

1. 按图结构建立基础方案 P_0 。
2. 在各 Core 的计算空闲区间中安排操作，得到方案 P_1 ；条件不满足时保留 P_0 。
3. 对 P_1 中选定的两个 Core 和最多 16 条链，按边界复制量求局部最小割。
4. 检查各 Core 各 Pipe 的计算时长上限；必要时固定部分链回到原位置后重算。
5. 只有 Step2 之前的复制量严格减少，才根据新分配重新安排顺序，得到 P_2 。
6. 保留未修改的 P_1 ，按 P_0 、 P_1 、 P_2 的构造顺序去掉完全相同的方案。
7. 完整评价至多三份方案，先比较 Makespan，再按代码规定比较新增 COPY；同分保留较早方案。

这里 P_0, P_1, P_2 是本次求解中产生的方案编号，不是问题编号。没有减少局部复制量，只能决定不产生 P_2 ，不能成为删除 P_1 的理由。

设图有 n 个计算操作、 m 条用于构造的计算依赖，查找空闲区间累计扫描量为 Q 。若共调整 R 个局部范围，第 j 个范围的辅助网络有 n_j 个节点、 m_j 条边，含 $w_j \leq 16$ 条可调整链，一次最小割耗时为 $T_{\text{cut}}(n_j, m_j)$ ，则该部分总耗时不超过

$$\sum_{j=1}^R (w_j + 1) T_{\text{cut}}(n_j, m_j).$$

此外还需计入依赖识别、排序、空闲区间扫描 Q 及汇合附近至多 K^2 个分配比较。局部最小割不调用完整评价程序，完整在线评分最多三次。全部步骤的时间均计入求解时间。

5.6 全部用例的结果

固定版本在全部 500 种图与 Core 数量组合上得到合法结果。官方主曲线保留使用一个 Core 时的加速比为 1，同时单独列出算法改善仅使用一个 Core 的方案后的比值。

Core 数量	1	2	3	4	5
官方口径平均加速比	1.0000	2.3167	3.2356	3.9711	4.5498
优化方案 $\text{mean}(B/M)$	1.2061	2.3167	3.2356	3.9711	4.5498
改善/持平/退化	59/41/0	64/36/0	54/46/0	49/51/0	42/58/0

与前一完整版本相比，268 种组合的 Makespan 缩短，232 种持平，没有变慢；69 张图至少在一个 Core 数量下得到改善。这些结果说明整个固定组合算法在给定数据集上的表现，不能单独证明每个局部方法对所有输入都不会退化。

全部组合的额外 DDR 总量从 3043860416 B 增至 4159738560 B，增加约 36.66%。其中 55 种组合的时间与搬运同时减少，199 种时间减少而搬运增加，14 种时间减少、搬运不变，232 种两项均不变。算法选择以完成时间为主，因此会接受这类以更多搬运换取更早完成的方案。

平均求解时间为 4.029 秒，95% 分位数为 18.841 秒，最大 40.187 秒。独立 E0 评价的平均时间为 1.410 秒，95% 分位数为 6.592 秒，最大 13.980 秒。这批结果在共享 Mac 上用个工作进程取得，整批约 46.63 分钟；新建进程不代表清空了操作系统缓存。

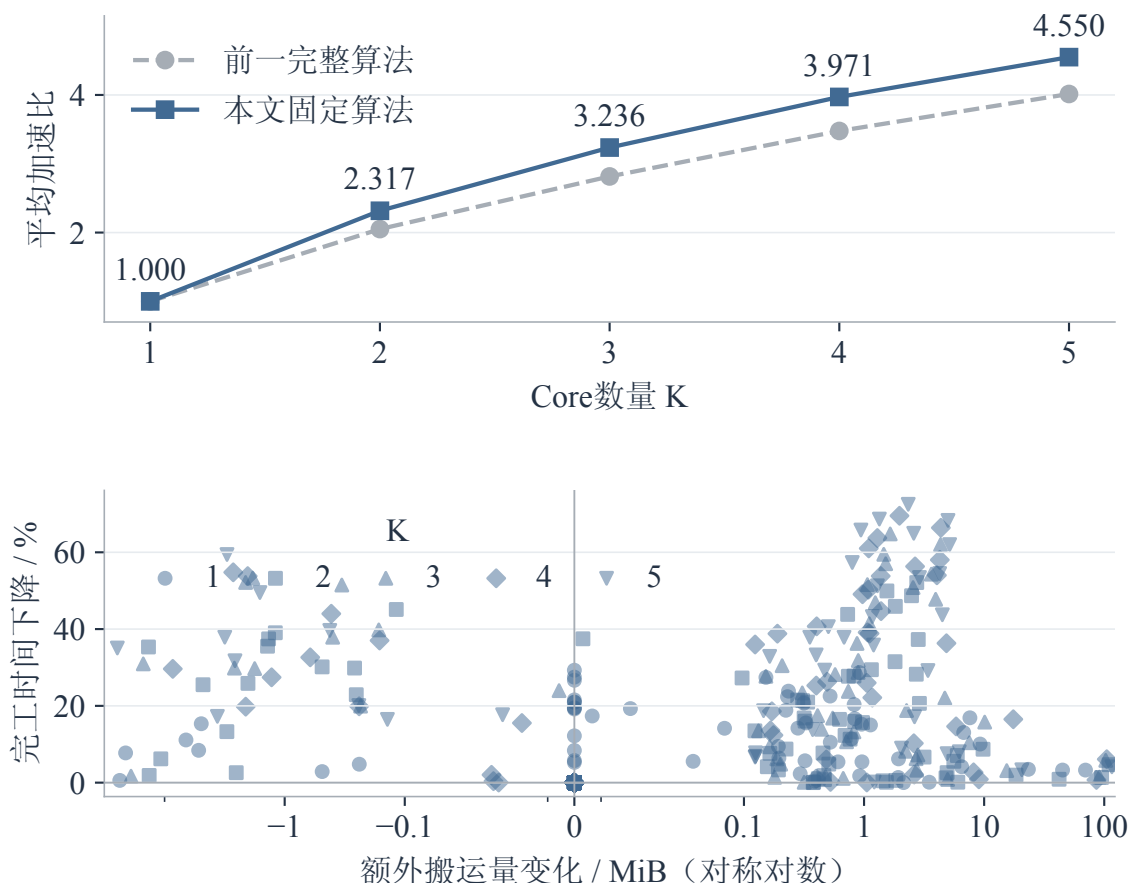


图 5.2 问题二的完整版本比较。上：按题目要求计算的平均加速比；下：500 种组合配对结果。部分时间改善伴随额外搬运增加，两项指标分别报告。

5.7 为什么没有额外缓存换入换出仍可能更慢

为检验只按容量和搬运量构造方案是否足够，已有实验对另一种固定构造 C04 评价了六个预先选择的 5 Core 用例。C04 是该实验的代码标识。六份方案都合法，且官方 Step2 未插入因容量不足产生的缓存换出、换入，但完成时间全部长于本章主算法。

case	主算法 / cycles	C04 / cycles	相对变慢
005	33515	38209	14.01%
016	2091375	2399125	14.72%
019	16247	28514	75.50%
025	942938	975658	3.47%
039	449653	494942	10.07%
075	343930	452120	31.46%

其中 case_019、case_039、case_075 的 COPY 字节减少，完成时间却增加。这个事实足以否定“少搬运且不发生缓存换入换出就必然更快”，但不能只凭两个汇总数字确定具体原因。即使计算操作的顺序不变，搬运操作的顺序、不同 Core 之间传递数据所需的等待及 DDR 请求同时发生的情况仍可能改变。

图 5.3 展示这两种方案的官方模拟时间线，说明搬运量较小并不意味着计算能够更早完成。

本章因此只用边界复制公式指导分配，不把它当作完整执行时间模型。问题三增加共享只读 Cache 后，还要判断读取请求到达时数据是否在 Cache 中，单看操作分到哪个 Core 更加不足。

6 问题三：安排计算顺序并利用共享只读 Cache

6.1 读取顺序为什么会改变 Cache 效果

问题三保留问题二的 Task 组成和分配到不同 Core 的操作之间的数据依赖，增加全部 Core 共享的只读 Cache。按官方代码，操作类型为 COPY_IN、能够确定搬入数据的逻辑标识且搬运字节数大于 0 时，操作开始就查询 Cache。程序使用这一逻辑标识查找数据。若数据存在，则为命中（cache hit），从 Cache 读取；否则为未命中（cache miss），从 DDR 读取，并在传输完成后把数据放入 Cache。FIFO 规则按插入时间选择需要移除的对象，命中不刷新该对象的次序。单份数据大于 Cache 容量时，不将它插入 Cache。

因此，前一次 DDR 传输尚未完成时，后续请求不能仅因数据正在传输就算作命中。即使命中，也仍须等待原有数据依赖及不同 Core 之间传递数据时的 500 cycles 等待条件满足，并占用 Cache 读取带宽。改变计算顺序会改变读取请求出现的时刻，而总命中字节数并不能完整描述这种时间关系。

本章算法在基础分配上，按输入结构尝试共享输入的访问顺序、符合条件的注意力计算顺序，以及下文定义的树状计算顺序。本文保留代码名称 Forest 以对应固定实现；这个名称只标识完整算法版本，不表示全部输入都是树。

例如，case_021、3 Core 的编译后张量 t1000000001 大小为 4096 B。官方事件给出以下生命周期；事件时刻为 cycles，表中间隔不表示等长处理阶段。

表 6.1 同一张量的 Cache 事件

时刻 / cycle	事件
0	COPY_IN 开始时未命中
207	DDR COPY_IN 完成，张量放入 Cache，记下插入顺序
4,956	另一 Core 读命中，插入次序不变
323,850	插入张量 t1000000487 时，该最早插入的项被移除

6.2 用一个小例子理解计算顺序与存储峰值

考虑两个互不依赖的计算分支 A、B。父操作必须等两者完成，才能读取它们的结果。A 先完成时，它留下的结果不能立刻释放，因为父操作还要等 B；这部分结果要与 B 计算期间的数据同时保留。反过来先计算 B，保留的数据量可能不同。

为了单独分析这个影响，先固定一棵树状计算结构：每个内部操作只输出一个张量，该结果只被唯一的父操作读取，其他树或分支不使用它。每次完整处理一棵子树后才处理下一棵，子树之间不交错执行。对子树 i 定义 h_i 为处理它时内部计算结果占用的最大总字

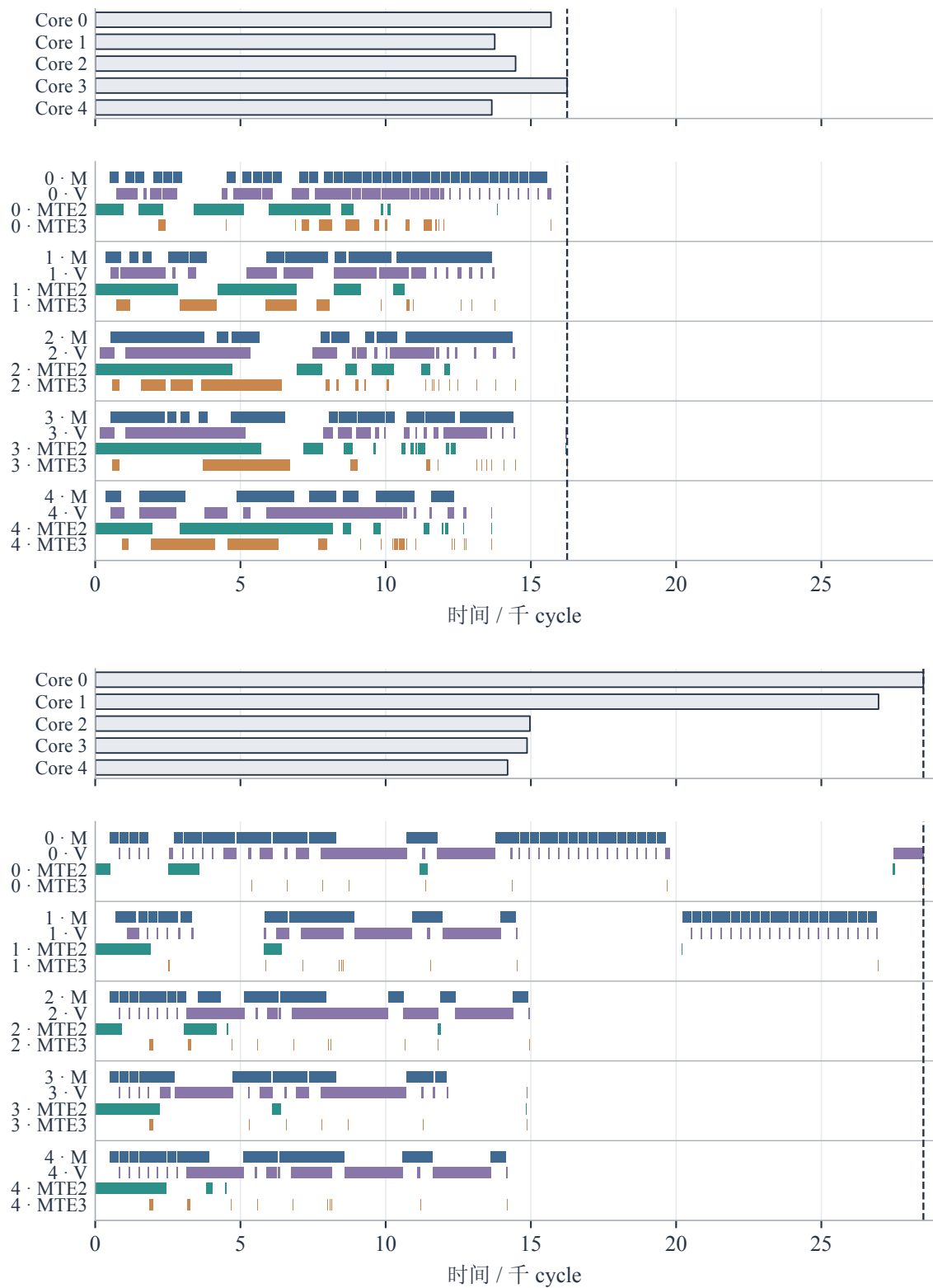


图 5.3 case_019、5 Core 的官方模拟的执行时间线。上：主算法， $M = 16,247$ cycles；下：C04， $M = 28,514$ cycles。每组分别显示 Task 和四条 Pipe，共用横轴范围。C04 的搬运量较少，且没有容量不足引起的缓存换入换出，但完成时间更长。

节数， r_i 为子树完成后必须保留的输出字节数。这里统计的是此模型内的结果数据，不包括外部输入，也不分别区分 L1 和 UB。

例如，A 的 $(h_A, r_A) = (8, 2)$ KiB，B 的 $(h_B, r_B) = (7, 5)$ KiB，父输出为 3 KiB。先 A 后 B 时，三个需要检查的量为 8、 $2 + 7 = 9$ 、 $2 + 5 + 3 = 10$ ，峰值为 10 KiB。先 B 后 A 时则为 7、 $5 + 8 = 13$ 、 $5 + 2 + 3 = 10$ ，峰值为 13 KiB。最后一项保留了父输出先分配、子结果随后释放的规则。这是公式示例，不是官方用例的存储测量。

图 6.1 按这三个处理步骤分解所占空间，保留的数据与正在处理的子树分别着色。

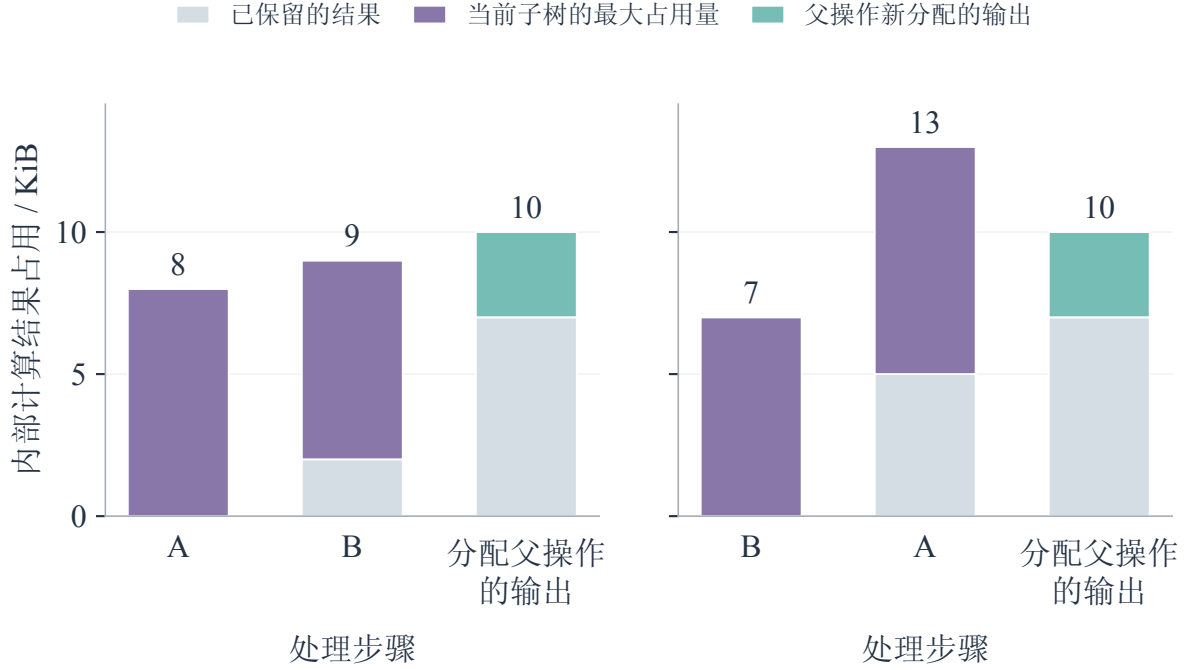


图 6.1 两种子树处理顺序的内部结果占用。左：先 A 后 B，峰值 10 KiB；右：先 B 后 A，峰值 13 KiB。横轴为处理步骤，不是模拟时钟；数字为第 6 章的公式示例。

6.3 证明怎样排序能够减小这部分存储峰值

设同一父操作有若干子树， $\pi = (\pi_1, \dots, \pi_m)$ 为它们的处理顺序，父输出大小为 r_{parent} 。当处理第 j 棵子树时，先前子树的输出共占用 $\sum_{\ell < j} r_{\pi_\ell}$ ，当前子树最多再占用 h_{π_j} 。父操作开始分配输出时，各子树输出仍需保留。因此峰值为

$$H(\pi) = \max \left\{ \max_j \left(\sum_{\ell < j} r_{\pi_\ell} + h_{\pi_j} \right), \sum_i r_i + r_{\text{parent}} \right\}.$$

所有项的单位均为 B。该式允许比较排序，却不直接包含执行时间。

命题 6.1。 若 $h_i \geq r_i \geq 0$ ，在上述不交错处理子树的模型中，按 $h_i - r_i$ 从大到小排序能够使 H 最小。

证明. 取相邻两棵子树 A、B，假设 $h_A - r_A \geq h_B - r_B$ 。暂时不计两者之前已经保留的共同结果，A 后 B 与 B 后 A 的最大占用分别为

$$H_{AB} = \max(h_A, r_A + h_B), \quad H_{BA} = \max(h_B, r_B + h_A).$$

由假设可得 $r_A + h_B \leq r_B + h_A$ ，又因 $r_B \geq 0$ 而有 $h_A \leq r_B + h_A$ ，故 $H_{AB} \leq H_{BA}$ 。之前保留的数据只会给两种次序各加同一个常数；两棵子树都完成后，各自留下的结果之和同为 $r_A + r_B$ ，后续计算及父输出分配不受交换影响。不断把次序不符合要求的相邻子树交换，最大占用不会增加，最终得到按 $h_i - r_i$ 递减的顺序。对于树自下而上应用这一结论即可。□

这种通过交换相邻分支证明存储次序的方法，与表达式树代码生成中的经典研究相关[4]。本章使用的是按字节给定大小的结果数据，不能把原文关于其他机器模型的结论直接当成本题证明。

算法先确认每个内部输出只被父操作读取、原 COPY 只位于允许的边界，并且依赖中的数据大小完整，再自下而上计算 h_i, r_i 。按所求顺序先处理子节点、后处理父节点，保持原来的 Core 分配，用单操作子图列表提交新顺序。

证明没有包括外部输入、不同树的最终结果共同保留、L1 与 UB 各自容量、编译后数据副本及官方实际分配顺序。因此，它只保证上述局部模型的最小峰值，不能保证全机器存储峰值最小、没有缓存换入换出或 Makespan 最小。新的顺序仍必须经过完整评价。

6.4 哪些候选可以提前排除

当一份候选的单操作子图、Core 分配和各 Core 各 Pipe 顺序已经固定时，可以列出每次成功执行都必须满足的时间不等式。对必要的先后关系 $u \rightarrow v$ ，写为

$$s_v \geq s_u + d_u + \delta_{uv}.$$

s_u, s_v 是开始时刻， d_u 是计算时长， δ_{uv} 只计入能够证明不可避免的等待，单位均为 cycle。同一对操作之间若有多个条件，应取最大的必要等待，不把重复条件相加。

将这些关系组成有向图，其最长路径长度 $L(P)$ 不大于固定候选的实际完成时间。如果当前已成功评价方案的完成时间为 U ，且 $L(P) \geq U$ ，该候选就不能严格改善，可以省去一次 E0 评价。

不能用“删去部分资源约束后，再运行一次贪心调度”的结果代替这个下界。约束减少只说明允许的方案更多，不保证某个固定贪心程序一定给出更短时间。本文采用的下界来自必要不等式，而不是另一个未证明最优的调度结果。

6.5 算法步骤与复杂度

算法 3：问题三的切分与调度。

输入：原图 G、Core 数量 K、官方固定配置。

1. 按结构与计算空闲区间构造基础方案，并进行评价。
2. 对满足条件的输入，尝试连续安排共享输入访问或调整注意力计算的次序。
3. 在剩余评价次数允许时，识别本章定义的树状计算，按子树峰值减去输出字节数的差值排序，产生方案。
4. 跳过与已有方案完全相同的结果；对满足证明条件的候选计算完成时间下界。
5. 若下界不小于当前最佳完成时间，则跳过完整评价。
6. 否则调用 E0，只有 Makespan 严格缩短才替换当前方案。
7. 一次求解共用至多三次 E0 预算，输出最佳成功方案及调用记录。

三次 E0 是整次求解的总限制，不是每种改动各可调用三次。若前面的候选已使用全部次数，后面的树排序就不会追加评价。结构不符合、方案重复、下界无法改善、合法性不通过和评价后未改善分别记录，避免把不同原因都写成“没有收益”。

给定含 n 个节点、 m 条依赖的树状结构，识别及按子先父后遍历需要 $O(n + m)$ 时间。每个父节点对孩子排序，合计需要 $O(\sum_v \deg(v) \log \deg(v))$ 时间，其中 $\deg(v)$ 为其孩子数。候选约束图含 m' 条必要边时，最长路径可在 $O(n + m')$ 时间内求得。完整算法还要计算基础分配并调用至多三次官方评价，不能用树排序的复杂度代表全过程。

6.6 同一方案启用和关闭 Cache 的结果

Forest 固定版本完成全部 500 种图与 Core 数量组合，均得到官方成功结果，500 次求解共调用 973 次在线 E0，没有重试。与前一完整策略相比，24 种组合的 Makespan 缩短、476 种持平，没有变慢。额外 COPY 和因容量不足产生的搬运分别有 18 种减少、479 种持平、3 种增加，两种统计各净减少 413450240 B。后者属于前者的一部分，不能相加为双倍减少量。

为比较 Cache 作用，对每份最终方案分别运行问题二、三的官方评价。下表“无 L2”是同一份 P3 方案关闭共享 Cache 后的结果，并不是问题二主算法生成的另一份方案。

Core 数量	无 L2: mean(B/M_2)	Cache: mean(B/M_3)	mean(M_2/M_3)	逐图字节命中率均值
1	1.1971	1.1998	1.0023	6.47%
2	2.2949	2.3120	1.0079	16.25%
3	3.1745	3.2587	1.0314	25.96%
4	3.9211	4.0915	1.0512	24.53%
5	4.4763	4.7576	1.0829	31.36%

5 个 Core 的逐图 Cache 加速比平均为 1.0829。这不表示所有图都缩短 8.29% 的时间，也不是每张图完成时间下降百分比的平均。按所有访问字节汇总后的命中率为 47.45%，逐图命中率算术平均则为 31.36%；前一种统计让访问字节较多的图占更大权重。

平均求解时间为 4.401 秒，中位数 1.616 秒，95% 分位数 20.009 秒，最大 51.335 秒。这些时间包含求解过程中调用 E0 的时间，来自共享主机四个工作进程的原始批次；后续实验采用不同进程数取得的时间没有替换进来。

6.7 额外的选择条件为什么可能拒绝更快方案

已有后续实验 R9F 对 5 Core 候选增加了三个同时满足的接受条件：启用 Cache 后的 M_3 严格下降，关闭 Cache 后的 M_2 不增加，且比值 $G = M_2/M_3$ 不下降。R9F 是该实验的代码标识，这三个条件不是官方评分要求。

100 张 5 Core 图中，96 张不符合该构造条件，1 张由候选下界排除，另外 3 张被上述联合条件拒绝，没有方案被接受。最终 500 份输出文件均与 Forest 一致。然而，被拒绝的三份候选实际上同时缩短了 M_2 和 M_3 ：

case/K	原 M_2	候选 M_2	原 M_3	候选 M_3
064/5	7029	6899	6981	6877
068/5	131631	98913	116345	97971
088/5	97305	66729	85789	66224

因为完成时间为正，保持 G 不下降等价于

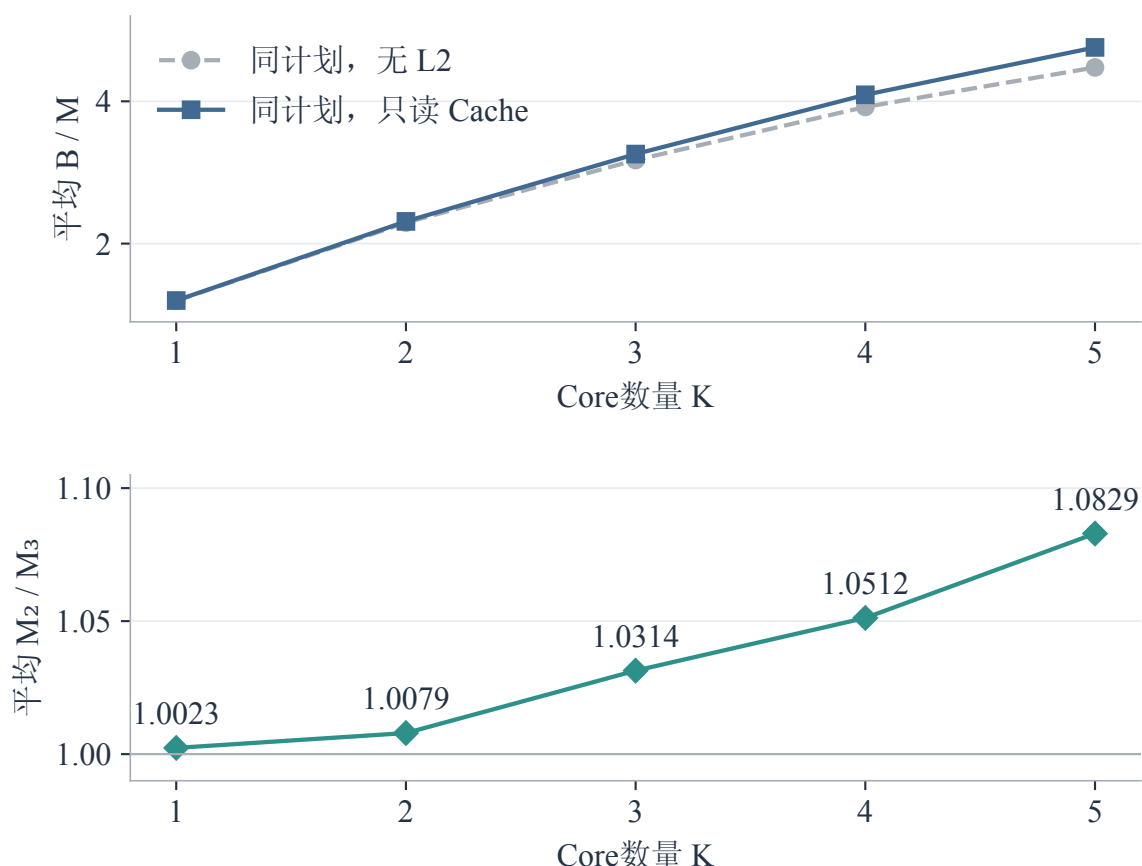


图 6.2 问题三的保持方案不变的配对评价。上：无 L2 与只读 Cache 的平均基线加速比；下：逐图 M_2/M_3 的均值，每种 Core 数量包含 100 对结果。

$$M_{2,\text{new}}M_{3,\text{old}} \geq M_{2,\text{old}}M_{3,\text{new}}.$$

当无 Cache 的时间改善得更快时，这个比值可能下降，即使启用 Cache 后的时间也明显缩短。因此，此实验说明额外的比值要求可能妨碍优化主要目标；不能把未接受解释为候选没有绝对收益。

主表仍保留 Forest 固定版本的完整结果。把上述三份方案手动替进主表，会成为依据已知结果逐例选择，不能代表已经完整评价的新算法。R9F 分两批执行 400 和 100 种组合，共使用 982 次在线 E0，仅用于说明这项额外选择条件的实际影响。

本章的结论是有条件的：所证排序能减少定义范围内的结果存储峰值，但 Cache 及完整执行效果仍需官方评价。保留这一边界，才能让局部证明、可执行算法与全量结果相互对应。

7 实验结果与分析

7.1 输入、对照和统计方法

实验采用题面给出的全部 100 张原始计算图，Core 数量分别取 1~5，每题共 500 种输入组合。三题各使用一个固定的完整求解器版本，方案构造条件和评价次数上限在运行前确定。每个结果对应的输入、程序版本、方案文件和官方评价结果都保留在清单中。本文没有把不同历史版本的逐例最佳结果拼成同一算法的成绩。

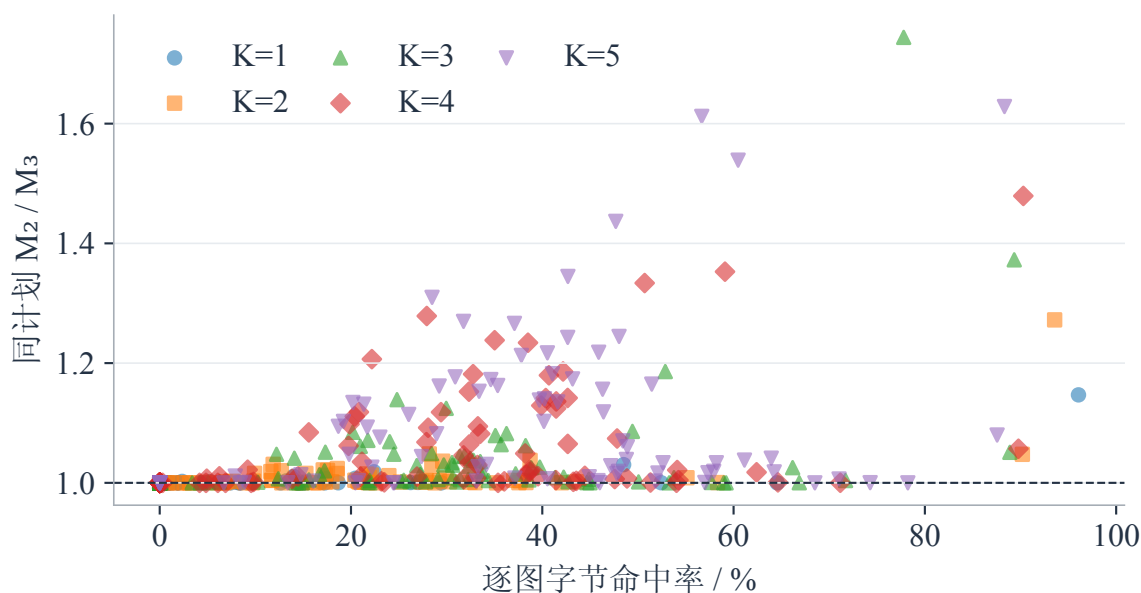


图 6.3 500 个保持方案不变的配对结果中的字节命中率与 Cache 收益。保留低于 1 的负收益点；100 个用例的不同 Core 数量结果相关，不能当作 500 个独立随机样本。

全文使用三类比较。第一类用官方仅使用一个 Core 时的基线除以当前方案的完成时间，得到题目要求的加速比。第二类比较相同题目、相同图和相同 Core 数量下两个完整算法版本，观察方法修改后的总体变化。第三类保持问题三的方案不变，只切换共享只读 Cache，比较该硬件条件对同一方案的影响。三类比较的分母不同，不能互相替换。

所有表格和图读取同一份 1500 行结果表。导出时检查每题 100 图与五种 Core 数量是否完整、基线是否对应原图、问题三的两份结果是否使用同一方案，再重新计算平均值。论文阶段读取已有结果，不重新运行求解器或官方评价程序。代码作者对方案与评价原件的核对记录另行保留，不能把静态表格检查等同于再次完成全部实验。

7.2 三问的主要结果

下表比较算法方案相对共同官方仅使用一个 Core 时的基线的逐图比值均值。为了展示算法还能怎样改变仅使用一个 Core 的方案，表内保留实际使用一个 Core 时的比值；问题一、二要求的主曲线仍按题面在使用一个 Core 时取 1。

Core 数量	P1 优化方案	P2 优化方案	P3 优化方案	P3 同计划无 L2
1	1.0021	1.2061	1.1998	1.1971
2	1.9516	2.3167	2.3120	2.2949
3	2.7514	3.2356	3.2587	3.1745
4	3.4786	3.9711	4.0915	3.9211
5	4.0553	4.5498	4.7576	4.4763

问题一要求不同 Task 之间的数据经过 DDR；问题二、三允许同一个 Core 保留在不同子图之间传递的数据，但仍受带宽、容量和不同 Core 之间传递数据所需的等待限制。表格呈现了这些场景下各自算法的结果，不能把两列之间的全部差值归因于一个特定机制。

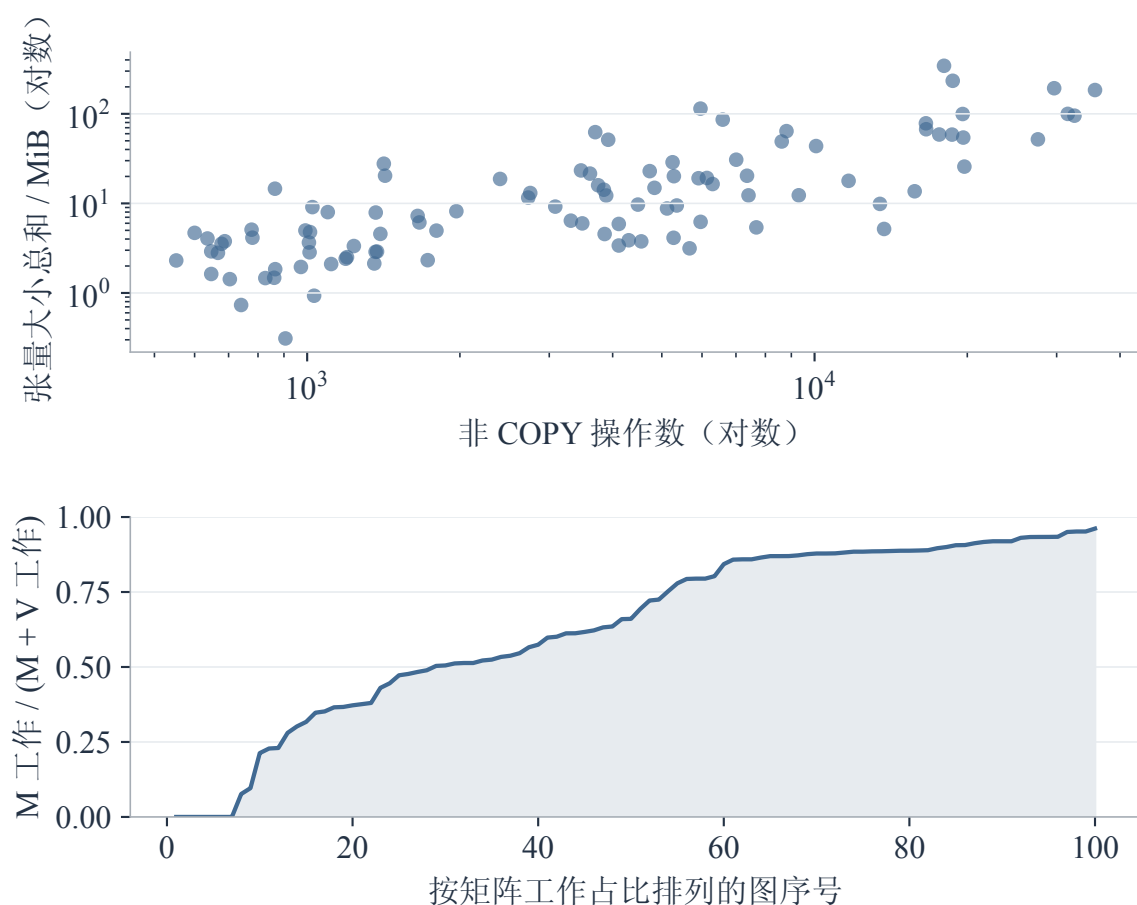


图 7.1 官方 100 张计算图的结构统计。上：非 COPY 操作数与张量大小总和；下：按矩阵计算的时长占两类计算总时长的比例排序的用例分布。

“P3 同计划无 L2” 一列使用问题三算法生成的方案；“P2 优化方案” 由问题二算法单独求得，两者不是同一个调度结果。5 Core 时，P3 两列平均基线加速比之比约为 1.0628，而逐图 Cache 加速比的均值是 1.0829。差异来自先求比值还是先求平均的顺序，本文以第 2 章定义的逐图比较为准。

7.3 生成方案需要多久

求解时间包含输入读取、构造与检查、在线方案评价及结果写出。问题一、二在求解过程中使用 E1、E2，最终 E0 复评另计；问题三在求解过程中直接使用 E0，其时间已经包含在求解总时间内。

方法与样本范围	在线评价总次数	求解均值 / s	P95 / s	最大 / s	运行条件
P1, 100 图、5 Core	全部 500 种组合 1130 次 E1	3.522	16.623	25.552	共享主机, 4 个工作进程
P2, 全部 500 种组合	1131 次 E2	4.029	18.841	40.187	共享主机, 1 个工作进程
P3 Forest, 全部 500 种组合	973 次 E0	4.401	20.009	51.335	共享主机, 4 个工作进程

表中的工作进程数说明同一批次同时运行多少个求解进程。主机还可能其他任务，因此不同批次的时间不能直接用作独占硬件的严格速度排名。问题一的时间一栏只统计 100 张图的 5 Core 结果，调用次数则覆盖全部 500 种组合，两者范围不同。

95% 分位数 (95th percentile, P95) 表示按时间排序后接近 95% 位置的值。问题一原始报告采用线性插值，问题二、三采用向上取整得到的第 $\lceil 0.95n \rceil$ 个次序值； n 是样本数量。本文保留各自实际报告的方法，不混合定义后声称细微的性能差别。

题面建议单个用例在 5~10 分钟内给出方案，这些成功进程在相应共享环境中的实测时间均低于这一量级。不过，本次实验没有进行独占硬件、清空操作系统缓存或跨平台重复测量，不能由此推断其他环境下的确定上限。

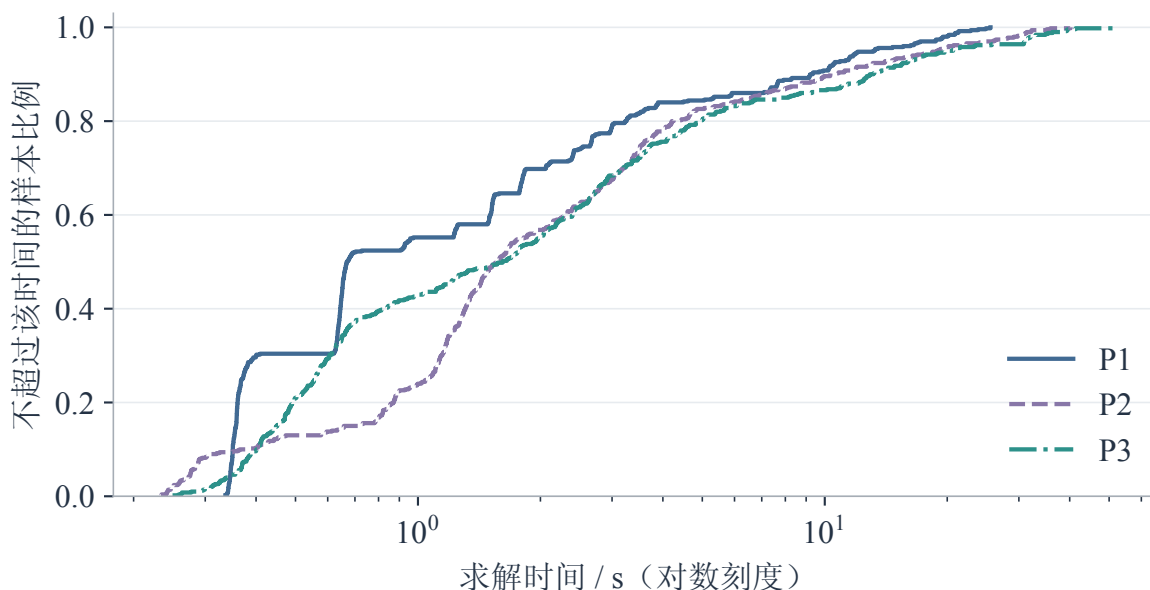


图 7.2 三种求解程序的实际经过时间累计分布，每题 500 次。时间包含方案构造及在线评价，横轴采用对数刻度；批次资源不同，不据此比较独占机器的速度。

7.4 哪些结论有实验支持

完整算法版本之间的比较，能说明修改后整体结果如何变化；某个构造被选中的记录，能说明该构造确实影响了输出；同一用例的结构图与时间线，能说明该次运行发生了什么。若多个部分同时修改，仅有前后版本比较还不能分别确定每一部分贡献多少。

问题一 5 Core 时平均加速比提高约 0.729%，额外 DDR 总量增加约 0.389%。问题二有 268 种组合缩短完成时间，但总额外 DDR 增加约 36.66%。这些结果共同说明“搬运更多”可以与“整体更快”同时出现；仅凭汇总数据，仍不能把全部改善归因于计算和搬运的时间重叠。

问题三完整版本有 24 种组合的时间改善，但有 3 种组合的额外 COPY 增加，因而不能声称所有指标同时改善。C04 六个预选用例全部变慢，说明该构造没有实现预期，却不证明所有减少搬运的方法都无效。R9F 则表明附加比值条件可能拒绝绝对完成时间更好的方案。本文保留这些结果，使算法选择的理由与限制同时可见。

7.5 下界能证明多接近最优

如果一个时间下界 L 适用于该输入的全部可行方案，而当前方案完成时间为 U ，则 $U/L - 1$ 给出相对最优差距的保守上限。问题二已有固定版本的下界核对记录：在 1~5 Core 下，分别有 81、62、32、33、23 张图满足 $U \leq 1.05L$ 。在记录所声明的条件下，这些结果距最优不超过 5%。这是逐输入的数学保证，不是统计显著性检验。

5 Core 时，平均 B/L 约为 6.108，高于算法达到的 4.550。下界忽略部分通信和容量约束时，差距也会增大，因此 6.108 不是已经证实可以达到的平均加速比。附录进一步分析工作量与必要等待怎样共同形成下界，但没有为这些新推导重新运行全量核对；本节仍使用已有固定结果。

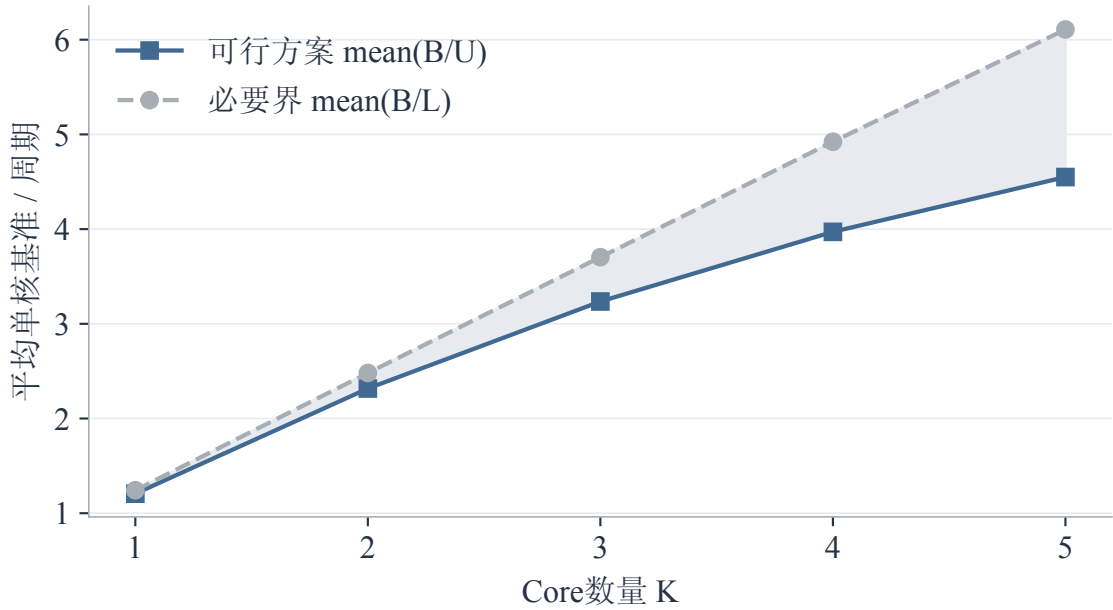


图 7.3 同一输入、配置与方案范围下的时间上界和下界。 $L \leq \text{OPT} \leq U$ ，阴影是必要界给出的上限与当前结果之间的差距，不代表一定可实现的收益。

7.6 实验范围与局限

全部 100 张官方图都参与过研发，结果只说明给定公开数据集上的表现，不构成对未见输入的验证。同一图的五种 Core 数量结果相互关联，不能直接当作 500 个独立随机样本估计显著性。按结构而非用例编号选择算法分支，能够避免显式按编号查表，但不能消除研发过程中接触公开图的影响。

当前三个主版本完成了规定的全部组合，不代表以后所有输入都一定成功。程序失败、超时、未运行应保留各自状态。最后，官方模拟为方案比较提供统一条件，但本次没有真实设备运行证据；本文不把模拟周期或本机求解秒数写成设备实测加速。

8 方法评价与结论

8.1 主要方法与适用范围

本文共同处理计算图切分、Core 分配和子图顺序三个决策。问题一通过重新分配独立计算支路增加并行机会，同时检查新增 Task 的等待与搬运。问题二根据生成、读取同一张

量的操作分布计算边界复制量，并在计算执行单元的空闲区间中安排操作。问题三对符合条件的树状计算选择较少占用内部结果存储空间顺序，再检查共享只读 Cache 条件下的完成时间。

三问的数学分析分别回答明确的问题。严格递增的子图编号证明新顺序无环；张量关联集合说明为什么同时移动多个读取操作可能减少复制，而逐个移动没有立即收益；相邻子树交换证明在限定模型中怎样排列能够减小存储峰值；完成时间的必要下界则排除不可能改善的固定候选。这些结论都有具体条件，不代替完整执行评价。

算法还有三方面限制。第一，共享带宽、缓存插入移除和存储空间分配会相互影响，目前的局部表示不足以完整预测全部执行行为。第二，树状计算排序等方法依赖特定输入结构；若一个结果被多个分支读取，或输入存在不满足公式要求的标识关系，就不能直接套用原证明。第三，已有实验仍发现局部更快但被附加条件拒绝的方案，表明接受规则还可以改进。修改后需要为新固定算法重新建立完整结果，不能把个别更好数字手动补进旧表。

8.2 结论

三个固定算法均完成 100 张图、每图 1~5 Core 的官方评价。5 Core 时，问题一、二、三的平均基线加速比分别为 4.0553、4.5498、4.7576；保持问题三方案不变，启用 Cache 相对关闭 Cache 的逐图平均加速比为 1.0829。结果来自统一输入与冻结配置，搬运增加和候选退化也一并报告。

这些结果支持本文的求解方法：先从原图依赖和数据使用关系构造少量方案，用证明检查必要条件，再以完整模拟决定性能是否改善。三问新增的硬件能力不同，但都需要明确区分数据量、执行时间和程序求解时间。后续工作将针对已出现的反例调整分配与接受规则，并在新的固定版本上验证。

参考文献

- [1] 第二十三届中国研究生数学建模竞赛. A 题：通用神经网络处理器下的多核调度问题及官方附件 [Z]. 2026.
- [2] GRAHAM R L. Bounds for Certain Multiprocessing Anomalies[J]. Bell System Technical Journal, 1966, 45(9): 1563–1581. DOI: 10.1002/j.1538-7305.1966.tb01709.x.
- [3] ÇATALYÜREK Ü V, AYKANAT C. Hypergraph-Partitioning-Based Decomposition for Parallel Sparse-Matrix Vector Multiplication[J]. IEEE Transactions on Parallel and Distributed Systems, 1999, 10(7): 673–693. DOI: 10.1109/71.780863.
- [4] SETHI R, ULLMAN J D. The Generation of Optimal Code for Arithmetic Expressions[J]. Journal of the ACM, 1970, 17(4): 715–728. DOI: 10.1145/321607.321620.
- [5] BLACK P E. Hyperedge; Hypergraph[EB/OL]. Dictionary of Algorithms and Data Structures, NIST. [2026-09-26]. <https://xlinux.nist.gov/dads/HTML/hyperedge.html>.

附录 B 补充数学推导

B.1 为什么可以保留计算依赖来求下界

先声明使用范围：每个需要讨论的原图中间张量至多由一个原始计算操作生成，不同原始张量标识不合并成同一逻辑数据标识。取出原始计算操作，只保留有依据的先后关系，包括原来就存在的计算关系，以及同一张量的生成操作必须先于读取操作的关系。不任意缩短经过原始 COPY 的路径。

在任何官方成功执行中，这些计算操作及其时长都被保留，每个 Core 的每条 Pipe 也仍同时至多处理一个操作。直接取该次执行中各计算的 Core 和开始、完成时刻，就得到满足保留关系的一组安排。因此，在这些必要关系下求得的最小必要时间，不会大于官方完成时间。

对于分配到不同 Core 的计算操作，必须核对两者之间的依赖怎样在官方执行中保留。问题一要求相关前驱 Task 完成后，后续 Task 才允许开始；问题二、三要求源 COPY_OUT 完成，再经过 500 cycles，目标 COPY_IN 才允许继续。接收端若因容量不足而需要再次读入数据，还受官方 Step2 次序和 MTE2 操作队列顺序限制，不能跳过所需的基础 COPY_IN。这里直接从成功执行中提取必要关系，不另用未经最优性证明的调度程序估计下界。

B.2 同一时间区间内必须完成多少计算

在 B.1 的计算依赖图上，为操作 v 定义两个不包括自身的时长： h_v 为它之前最长依赖路径的时长， t_v 为它之后最长依赖路径的时长。无相应前驱或后继时取 0。递推式为

$$h_v = \max_{u \rightarrow v} (h_u + d_u), \quad t_v = \max_{v \rightarrow w} (d_w + t_w).$$

本节 h_v 以 cycle 计，与第 6 章按字节定义的子树存储量 h_i 是不同的局部记号。取 $a, b \geq 0$ ，选定一种 Pipe p ，考虑非空操作集合

$$S = \{v : p_v = p, h_v \geq a, t_v \geq b\}.$$

任何成功执行中， S 的操作都只能完整安排在 $[a, M - b]$ 内：它们之前至少需要 a 个周期，之后至少需要 b 个周期。同类 Pipe 共有 K 个，每个操作又不能拆成多段交给不同 Core。因此，除了总工作量，还可以利用操作不可分割这一条件。

将 S 中 n 个操作的时长降序排列为 $d_{(1)} \geq \dots \geq d_{(n)}$ 。定义

$$A_K(S) = \max_{\substack{r \geq 1 \\ m = (r-1)K + 1 \leq n}} \sum_{j=m-r+1}^m d_{(j)}.$$

于是

$$M \geq a + b + \max \left\{ \left\lceil \frac{\sum_{v \in S} d_v}{K} \right\rceil, A_K(S) \right\}.$$

证明. 令 $x = M - a - b$ ， K 个同类 Pipe 在该区间内最多提供 Kx 个计算周期，得到第一项。取前 $m = (r-1)K + 1$ 个操作，至少一个 Core 要执行其中 r 个；即使选其中最长的 r 个，也至少需要 $\sum_{j=m-r+1}^m d_{(j)}$ 周期。对允许的 r 取最大得到第二项。两项都限制同一个区间的长度，故取最大值，不相加。最后只加一次前后所需的 $a + b$ 。□

例如，两个 Core 执行三个不可拆分操作，时长分别为 10、8、7 cycles。总工作量给出 13 cycles 的下界，上式利用不可分割性给出 15 cycles。达到 15 仍可能受其他依赖和资源限制，这个例子只说明下界为何能加强。

图上递推 h_v, t_v 需要 $O(n + m)$ 时间；固定 a, b 后，检查集合成员需 $O(n)$ ，对选中操作排序需 $O(|S| \log |S|)$ 。枚举所有 a, b 还要另计成本。若 S 为空，不能仅凭 $a + b$ 得到有效下界。

B.3 怎样避免重复计算不同 Core 之间传递数据所需的等待

考虑一串前后有依赖的特定计算操作。要求每个所选操作与其间全部相关计算都存在有向路径上的先后关系，从而将其他计算放入前后两个所选操作之间、互不重叠的时间区间。只有满足这一条件，后文才能把各区间的必要时间相加；任取几个汇合节点并不足够。

对已证明必须经过的分配到不同 Core 的计算操作之间的依赖，等待常数记为 τ_q ，问题一取 1000，问题二、三取 500 cycles。某个区间内选取 B.2 的非空使用同一种 Pipe 的操作集合，前后必要计算时长为 a, b 。令 $\ell, r \in \{0, 1\}$ 分别表示区间前后是否有所选约束操作。记 $(x)_+ = \max(x, 0)$ ，扣除前后计算已经覆盖的时间后，仍需考虑的等待为

$$c(a, b) = \ell(\tau_q - a)_+ + r(\tau_q - b)_+.$$

设整个时间区间长度为 x ，可安排集合中操作的时间为 $z = x - a - b$ ，总计算工作量为 $W(S) = \sum_{v \in S} d_v$ 。若前后两个所选操作在同一 Core，该 Core 最多可使用 z ，其他 Core 各至多可使用 $(z - c)_+$ ，故

$$W(S) \leq z + (K - 1)(z - c)_+.$$

若前后操作位于不同 Core，必有 $K \geq 2$ 。令 $A = (\tau_q - a)_+$ 、 $B = (\tau_q - b)_+$ ，可供同类计算使用的总周期数至多为

$$(z - A)_+ + (z - B)_+ + (K - 2)(z - A - B)_+.$$

逐段比较 z 与 $A, B, A + B$ 可得

$$(z - A)_+ + (z - B)_+ \leq z + (z - A - B)_+.$$

所以仍可采用前式限制工作量。对整数周期，满足 $W \leq z + (K - 1)(z - c)_+$ 所需的最小 z 为

$$\phi_{K,c}(W) = \min \left\{ W, \left\lceil \frac{W + (K - 1)c}{K} \right\rceil \right\}.$$

当 $W \leq c$ 时， $z = W$ 满足第一段；当 $W > c$ 时，解第二段线性不等式并向上取整。结合操作不可分割的限制，得到

$$x \geq a + b + \max\{\phi_{K,c(a,b)}(W(S)), A_K(S)\}.$$

满足开头条件时，不同区间及分隔计算互不重叠，可以把各自己证下界相加。如果只是从几个相互重叠的窗口得到下界，就只能取最大值，不能重复累计。问题一的 100 cycles 在同一个 Core 中切换 Task 的等待也不能自动加入，因为方案未必在所选计算处划分 Task。

这段推导说明应扣除已有计算时长覆盖的等待，不能把 $a + b$ 、不同 Core 之间传递数据所需的全部等待时间和工作量简单相加。它属于补充理论分析，尚未据此生成新的全量实验结果，正文的下界统计仍取原固定记录。

B.4 少量状态信息怎样保证后续判断不变

本节讨论一个尚未由本文主算法一般实现的问题：能否只保存方案的一部分信息，就正确判断后续修改及其结果。两份方案当前 **Makespan** 相同，并不足以说明这一点。例如，它们可能采用不同的 **Core** 顺序，同一次移动操作在一份方案中合法，在另一份中却造成循环依赖。

先固定允许的修改集合 $\mathcal{A}$ ，例如在已经声明的结构范围内移动、合并或拆分子图；每种修改的参数含义也固定。用 D_a 表示允许执行修改 a 的方案集合，用 $\Phi(P)$ 表示需要保留的观测结果。若只研究成功执行，可取观测为 **Makespan**；若还包含失败，则须把成功或失败的状态纳入观测，不能把未定义的 **Makespan** 填成一个周期数。

令 $\mathcal{L}(P)$ 为从方案 P 出发能够连续执行的所有有限修改序列，包含不作修改的空序列。两份方案在本节所关心的意义下等价，要求

$$\mathcal{L}(P) = \mathcal{L}(Q), \quad \Phi(w(P)) = \Phi(w(Q)), \quad \forall w \in \mathcal{L}(P).$$

其中 $w(P)$ 表示按序列 w 依次修改 P 。第一个条件保证可做的修改完全一致，第二个保证修改后的观测一致；只比较双方碰巧都能做的修改会漏掉第一类差别。

命题 B.1. 设 $S(P)$ 为准备保存的信息。对每种修改 a ，若存在判定函数 d_a 、更新函数 $\bar{a}$ 和观测恢复函数 o ，使声明范围内的全部方案满足

$$P \in D_a \iff d_a(S(P)), \quad S(a(P)) = \bar{a}(S(P)), \quad \Phi(P) = o(S(P)),$$

且更新等式在 a 可执行时成立，则 $S(P) = S(Q)$ 蕴含上述等价关系。

证明. 不作修改时，由观测恢复等式知两份观测相同。若执行相同的前 j 步后保留的信息仍相同，则 d_a 保证下一步在两份方案中同时允许或同时不允许；允许时， $\bar{a}$ 使更新后的信息仍相同， o 再恢复相同观测。对序列长度归纳，即得全部可执行序列与后续观测一致。□

命题给出充分条件，没有证明保存的信息已经最少。要说明最少，还需证明不同保存状态必能被某个后续修改或观测区分。把整份方案编码成一个整数没有减少信息量；重复使用相同编译结果、保存程序运行现场、删除完全相同的候选，则分别解决较小的问题，不能自动获得本命题关于所有后续修改的保证。

附录 C 固定实现与复现信息

三题固定求解器的提交分别为下列 SHA。逐输入的图、方案与结果来源及完整 SHA-256 见配套 CSV 与来源收据。固定版本结果不能用不同算法的历史赢家拼接。

- P1: 834d8c957538ee069c66aadac9509552a4cc69d7；求解过程中的评分至多 10 次 E1。
- P2: c66559a6f8a31ef7b4720e1f7c3c28d61f8dff3f；求解过程中的评分至多 3 次 E2。
- P3: 311322b996c0948e8a6a9c7ec6ddfe6ae41fbee1；求解过程中的评分至多 3 次 E0。

官方代码与配置身份分别为 de11a83db8d7c47ed328b15a7df71d613a833b16cd23ee9fe877999578a1ace0 和 dcd10de54b23f8366428fb24e828812b1da9549e6eae4a3c3f38604fe5ae77b9。来源收据记录 822 个固定 Git 对象；本稿的数据导出仅读取已有原件，没有新增求解器或评价器调用。